PYTHON PROGRAMMING — ACTIVITY-BASED LEARNING RECORD

First 10 Python Case Studies

Algorithms · ANSI Flowcharts · Pseudocode · Source Code · Test Case & Output Screenshots

Syllabus Scope: Variables, Data Types, Operators, Conditionals, Loops, Strings, Lists, Nested Lists, Tuples & Unpacking

GitHub Deliverable Repository: Python-Activity-Based-Learning

SUBMISSION DETAILS

Student Name:	**Student Submission Copy**	Evaluating Faculty:	**Department of Computer Science &**
Registration / Roll No.:	**REG-2026-PY01**	Academic Session:	**2026–2027**
Branch & Section:	**B.Tech CSE — Semester I**	Evaluation Weightage:	**10 Marks (All 10 Deliverables)**

MANDATORY DELIVERABLES INCLUDED FOR EVERY QUESTION:

[x] Problem Statement [x] Objectives [x] Assumptions [x] Step-by-Step Algorithm [x] ANSI Vector Flowchart
[x] Structured Pseudocode [x] Syllabus-Only Python Code [x] Test Cases & Input Screenshots [x] Output Screenshots [x] Conclusion

INDEX OF SELECTED CASE STUDIES

Q#	Case Study Title	Core Syllabus Constructs Applied	Source File
Q01	**University Scholarship Allocation**	Tuples & Unpacking · Nested Loops (Bubble	`case_study_01.py`
Q02	**Progressive Electricity Billing**	Multi-Branch elif Slabs · Tuple Records	`case_study_02.py`
Q03	**Hospital Billing and Patient Classification**	Input Constraint Validation · Nested	`case_study_03.py`
Q04	**Employee Payroll and Deduction Analysis**	Progressive Tax Slabs · Boundary Clamping	`case_study_04.py`
Q05	**Multi-Plan Telecommunications Cost Optimization**	Tuple Catalog Traversal · Constrained	`case_study_05.py`
Q06	**Railway Fare and Reservation Calculator**	Distance Slabs · Tuple Unpacking	`case_study_06.py`
Q07	**Retail Billing with Multiple Discount Rules**	Nested Lists · Multi-Stage Discounts	`case_study_07.py`
Q08	**Manufacturing Cost and Profitability Analysis**	Cost Accounting Formulas · Tuple Unpacking	`case_study_08.py`
Q09	**Multi-Currency Travel Budget Analysis**	Nested List Aggregation (Dict-Free) · Tuple	`case_study_09.py`
Q10	**Campus Event Financial Planning**	Fixed vs Variable Costing · While-Loop	`case_study_10.py`

CASE STUDY 01 OF 10

1. University Scholarship Allocation

1. Problem Statement

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

2. Objectives

- Store multi-attribute student application records using nested tuples and lists without dictionaries.
- Unpack applicant tuples and nested 3-subject mark tuples to compute aggregate percentages.
- Validate four simultaneous eligibility thresholds: aggregate >= 75%, all subjects >= 60, attendance >= 80%, and annual income <= Rs. 5,00,000.
- Rank eligible candidates via nested-loop comparison resolving ties deterministically by aggregate, attendance, and lower family income.

3. Assumptions & Domain Rules

- Each applicant record is stored as (app_id, name, (m1, m2, m3), attendance_pct, annual_income).
- Minimum eligibility requires Aggregate >= 75.0%, each individual subject score >= 60.0, Attendance >= 80.0%, and Annual Income <= Rs. 5,00,000.
- Tie-breaking priority order: (1) Higher Aggregate %, (2) Higher Attendance %, (3) Lower Annual Family Income.
- Scholarship tiers assigned by rank: Rank 1 receives 100% Waiver, Rank 2 receives 75% Waiver, and Rank 3+ receives 50% Merit Grant.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read total number of applicants n and initialize empty lists applicants, eligible, and rejected.

Step 3: Repeat n times: read app_id, name, subject marks m1, m2, m3, attendance, and income; append tuple (app_id, name, (m1, m2, m3), attendance, income) to applicants.

Step 4: Traverse each tuple in applicants using tuple unpacking to extract app_id, name, (s1, s2, s3), attendance, and income.

Step 5: Compute aggregate = (s1 + s2 + s3) / 3.0 and check all four eligibility conditions.

Step 6: If all conditions hold, append (app_id, name, aggregate, attendance, income) to eligible; otherwise concatenate failed reasons and append to rejected.

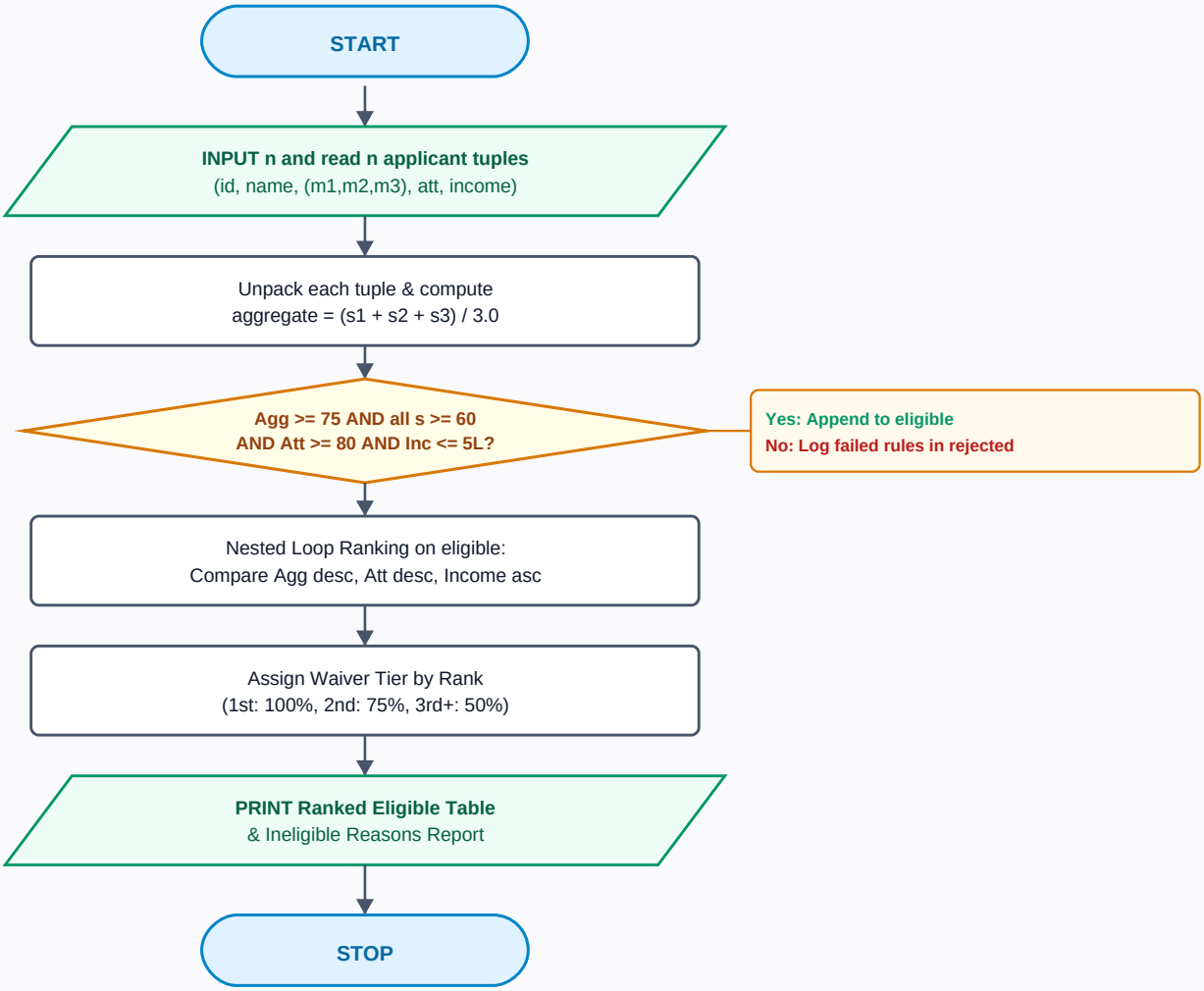
Step 7: Sort eligible list using nested loops (i from 0 to count-1, j from 0 to count-i-2): swap adjacent tuples if next candidate has higher aggregate, or equal aggregate with higher attendance, or both equal with lower income.

Step 8: Assign scholarship tier by final rank (Rank 1 -> 100% Waiver, Rank 2 -> 75% Waiver, Rank >= 3 -> 50% Merit) and print formatted eligible and rejected tables.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 01: University Scholarship Allocation

```
BEGIN
    INPUT n
    SET applicants = [], eligible = [], rejected = []
    FOR i = 1 TO n DO
        INPUT app_id, name, m1, m2, m3, attendance, income
        APPEND (app_id, name, (m1, m2, m3), attendance, income) TO applicants
    END FOR

    FOR EACH applicant IN applicants DO
        UNPACK (app_id, name, (s1, s2, s3), attendance, income) = applicant
        SET aggregate = (s1 + s2 + s3) / 3.0
        IF aggregate >= 75 AND s1 >= 60 AND s2 >= 60 AND s3 >= 60 AND attendance >= 80 AND
     income <= 500000 THEN
            APPEND (app_id, name, aggregate, attendance, income) TO eligible
        ELSE
            BUILD reason_string OF unmet conditions
            APPEND (app_id, name, aggregate, reason_string) TO rejected
        END IF
    END FOR
```

PSEUDOCODE — Case Study 01: University Scholarship Allocation (continued — part 2)

```
    FOR i = 0 TO LENGTH(eligible) - 1 DO
        FOR j = 0 TO LENGTH(eligible) - i - 2 DO
            IF eligible[j+1] > eligible[j] BY (aggregate DESC, attendance DESC, income ASC
    ) THEN
                SWAP eligible[j] AND eligible[j+1]
            END IF
        END FOR
    END FOR

    PRINT Formatted Ranked Scholarship Table AND Rejected List
END
```

7. Python Source Code (case_study_01.py)

case_study_01.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 1: University Scholarship Allocation
# Syllabus Concepts: Variables, Operators, Conditionals, Nested Loops, Strings, Lists,
     Tuples, Unpacking

n = int(input("Enter number of applicants: "))
applicants = []

for i in range(n):
    app_id = input("Applicant ID: ").strip()
    name = input("Student Name: ").strip()
    m1 = float(input("Subject 1 Marks: "))
    m2 = float(input("Subject 2 Marks: "))
    m3 = float(input("Subject 3 Marks: "))
    attendance = float(input("Attendance (%): "))
    income = float(input("Annual Family Income (INR): "))
    record = (app_id, name, (m1, m2, m3), attendance, income)
    applicants.append(record)

eligible = []
rejected = []

for applicant in applicants:
    app_id, name, marks, attendance, income = applicant
    s1, s2, s3 = marks
    aggregate = (s1 + s2 + s3) / 3.0

    reasons = []
    if aggregate < 75.0:
        reasons.append("Aggregate < 75%")
    if s1 < 60.0 or s2 < 60.0 or s3 < 60.0:
        reasons.append("Subject < 60")
    if attendance < 80.0:
        reasons.append("Attendance < 80%")
    if income > 500000.0:
        reasons.append("Income > 500000")

    if len(reasons) == 0:
        eligible.append((app_id, name, aggregate, attendance, income))
    else:
        reason_text = ""
        for idx in range(len(reasons)):
            if idx > 0:
                reason_text = reason_text + ", "
            reason_text = reason_text + reasons[idx]
        rejected.append((app_id, name, aggregate, reason_text))

# Rank eligible students using nested loops and tie-breaking rules:
# 1. Higher aggregate  2. Higher attendance  3. Lower family income
count = len(eligible)
```

case_study_01.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
for i in range(count):
    for j in range(0, count - i - 1):
        id_a, name_a, agg_a, att_a, inc_a = eligible[j]
        id_b, name_b, agg_b, att_b, inc_b = eligible[j + 1]
        swap = False
        if agg_b > agg_a:
            swap = True
        elif agg_b == agg_a and att_b > att_a:
            swap = True
        elif agg_b == agg_a and att_b == att_a and inc_b < inc_a:
            swap = True
        if swap:
            eligible[j], eligible[j + 1] = eligible[j + 1], eligible[j]

print("\n" + "=" * 68)
print("          UNIVERSITY SCHOLARSHIP ALLOCATION REPORT")
print("=" * 68)
print("ELIGIBLE & RANKED CANDIDATES:")
print("-" * 68)
if len(eligible) == 0:
    print("No eligible candidates found.")
else:
    print(f"{'Rank':<6}{'ID':<8}{'Name':<14}{'Agg(%)':<10}{'Att(%)':<10}{'Income':<11}
    {'Tier'}")
    print("-" * 68)
    for rank_idx in range(len(eligible)):
        app_id, name, agg, att, inc = eligible[rank_idx]
        rank_no = rank_idx + 1
        if rank_no == 1:
            tier = "100% Waiver"
        elif rank_no == 2:
            tier = "75% Waiver"
        else:
            tier = "50% Merit"
        print(f"{rank_no:<6}{app_id:<8}{name:<14}{agg:<10.2f}{att:<10.1f}{inc:<11.0f}{
    tier}")

print("-" * 68)
print("INELIGIBLE APPLICANTS:")
if len(rejected) == 0:
    print("None")
else:
    for item in rejected:
        r_id, r_name, r_agg, r_reason = item
        print(f"{r_id:<8}{r_name:<14}Agg: {r_agg:6.2f}% | Failed: {r_reason}")
print("=" * 68)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-01A	Tie-Break Resolution	S101 (Agg 88%, Att 92%, Inc 3.2L) vs S102 (Agg 88%, Att 92%, Inc 2.4L)	S102 Ranked #1 (100% Waiver), S101 Ranked #2 (75% Waiver)	[PASS] Rank 1: S102 Rohan | Rank 2: S101 Ananya
TC-01B	Subject Minimum Guard	S103 Meera: Marks (90, 58, 92), Agg 80.0%, Att 89%	Rejected due to Subject < 60 despite 80% aggregate	[PASS] Ineligible: Failed Subject < 60
TC-01C	Zero Eligible Edge Case	S201 (Agg 72%), S202 (Att 76%)	No eligible candidates found; both listed with failure reasons	[PASS] No eligible candidates found; 2 rejected

Test Case 1: Tie-Breaking & Multi-Rule Rejection — Input Session Screenshot

4 applicants: S101 & S102 tie on Aggregate (88.0%) and Attendance (92.0%) so lower income (S102: Rs. 2,40,000) wins Rank 1; S103 fails single-subject minimum (58 < 60); S104 qualifies Rank 3.

Terminal — Test Case 1 Input Screenshot (python3 case_study_01.py)

```
$ python3 case_study_01.py
Enter number of applicants: 4
Applicant ID: S101
Student Name: Ananya
Subject 1 Marks: 88
Subject 2 Marks: 91
Subject 3 Marks: 85
Attendance (%): 92
Annual Family Income (INR): 320000
Applicant ID: S102
Student Name: Rohan
Subject 1 Marks: 88
Subject 2 Marks: 85
Subject 3 Marks: 91
Attendance (%): 92
Annual Family Income (INR): 240000
Applicant ID: S103
Student Name: Meera
Subject 1 Marks: 90
Subject 2 Marks: 58
Subject 3 Marks: 92
Attendance (%): 89
Annual Family Income (INR): 280000
Applicant ID: S104
Student Name: Vikram
Subject 1 Marks: 79
Subject 2 Marks: 82
Subject 3 Marks: 80
Attendance (%): 85
Annual Family Income (INR): 410000
```

Test Case 2: All Applicants Ineligible Edge Case — Input Session Screenshot

2 applicants where S201 fails minimum aggregate (72.0% < 75%) and S202 fails minimum attendance (76.0% < 80%). Verifies graceful empty-ranking report.

Terminal — Test Case 2 Input Screenshot (python3 case_study_01.py)

```
$ python3 case_study_01.py
Enter number of applicants: 2
Applicant ID: S201
Student Name: Kavya
Subject 1 Marks: 72
Subject 2 Marks: 74
Subject 3 Marks: 70
Attendance (%): 88
Annual Family Income (INR): 300000
Applicant ID: S202
Student Name: Arjun
Subject 1 Marks: 84
Subject 2 Marks: 86
Subject 3 Marks: 82
Attendance (%): 76
Annual Family Income (INR): 250000
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_01.py)

```
======================================================================
          UNIVERSITY SCHOLARSHIP ALLOCATION REPORT
======================================================================
ELIGIBLE & RANKED CANDIDATES:
----------------------------------------------------------------------
Rank  ID      Name          Agg(%)    Att(%)    Income     Tier
----------------------------------------------------------------------
1     S102    Rohan         88.00     92.0      240000     100% Waiver
2     S101    Ananya        88.00     92.0      320000     75% Waiver
```

Terminal — Test Case 1 Verified Output Screenshot (case_study_01.py) (continued — part 2)

```
3     S104    Vikram        80.33     85.0      410000     50% Merit
----------------------------------------------------------------------
INELIGIBLE APPLICANTS:
S103    Meera         Agg:  80.00% | Failed: Subject < 60
======================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_01.py)

```
======================================================================
          UNIVERSITY SCHOLARSHIP ALLOCATION REPORT
======================================================================
ELIGIBLE & RANKED CANDIDATES:
----------------------------------------------------------------------
No eligible candidates found.
----------------------------------------------------------------------
INELIGIBLE APPLICANTS:
S201    Kavya         Agg:  72.00% | Failed: Aggregate < 75%
S202    Arjun         Agg:  84.00% | Failed: Attendance < 80%
======================================================================
```

10. Conclusion

The University Scholarship Allocation system successfully evaluates multi-criteria academic and financial constraints using nested tuples, tuple unpacking, and nested-loop bubble ranking without relying on external libraries or built-in sort key functions.

CASE STUDY 02 OF 10

2. Progressive Electricity Billing

1. Problem Statement

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

2. Objectives

- Compute tiered domestic energy charges across four progressive kWh consumption slabs.
- Apply fixed meter charges, low-usage conservation rebates (<= 100 units), and high-load surcharges (> 400 units).
- Track multi-month billing tuples sequentially to compute month-over-month percentage changes.
- Flag abnormal billing spikes whenever monthly net bill increases by more than 25.0% over the preceding month.

3. Assumptions & Domain Rules

- Consumption slabs: 0–100 units @ Rs. 3.00/u, 101–200 @ Rs. 5.00/u, 201–400 @ Rs. 7.00/u, and >400 @ Rs. 9.50/u.
- Fixed monthly meter rent is Rs. 75.00 for all active connections.
- Conservation Rebate: Rs. 50.00 off energy charge if consumption <= 100 units; High-Load Surcharge: 10% on (energy + fixed) if consumption > 400 units.
- Month 1 serves as BASELINE; any subsequent month with > 25.0% bill growth over the immediate prior month is flagged as SPIKE.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read consumer_id, consumer_name, and number of months num_months.

Step 3: Loop num_months times to input (month_label, units) tuples into monthly_records.

Step 4: For each (month_label, units) tuple, clamp negative units to 0 and evaluate progressive slab energy_charge using if-elif-else.

Step 5: Set fixed_charge = 75.0. If units <= 100, apply rebate = min(50.0, energy_charge). Else if units > 400, apply surcharge = 0.10 * (energy_charge + fixed_charge).

Step 6: Calculate net_bill = energy_charge + fixed_charge - rebate + surcharge and append billing tuple to computed_bills.

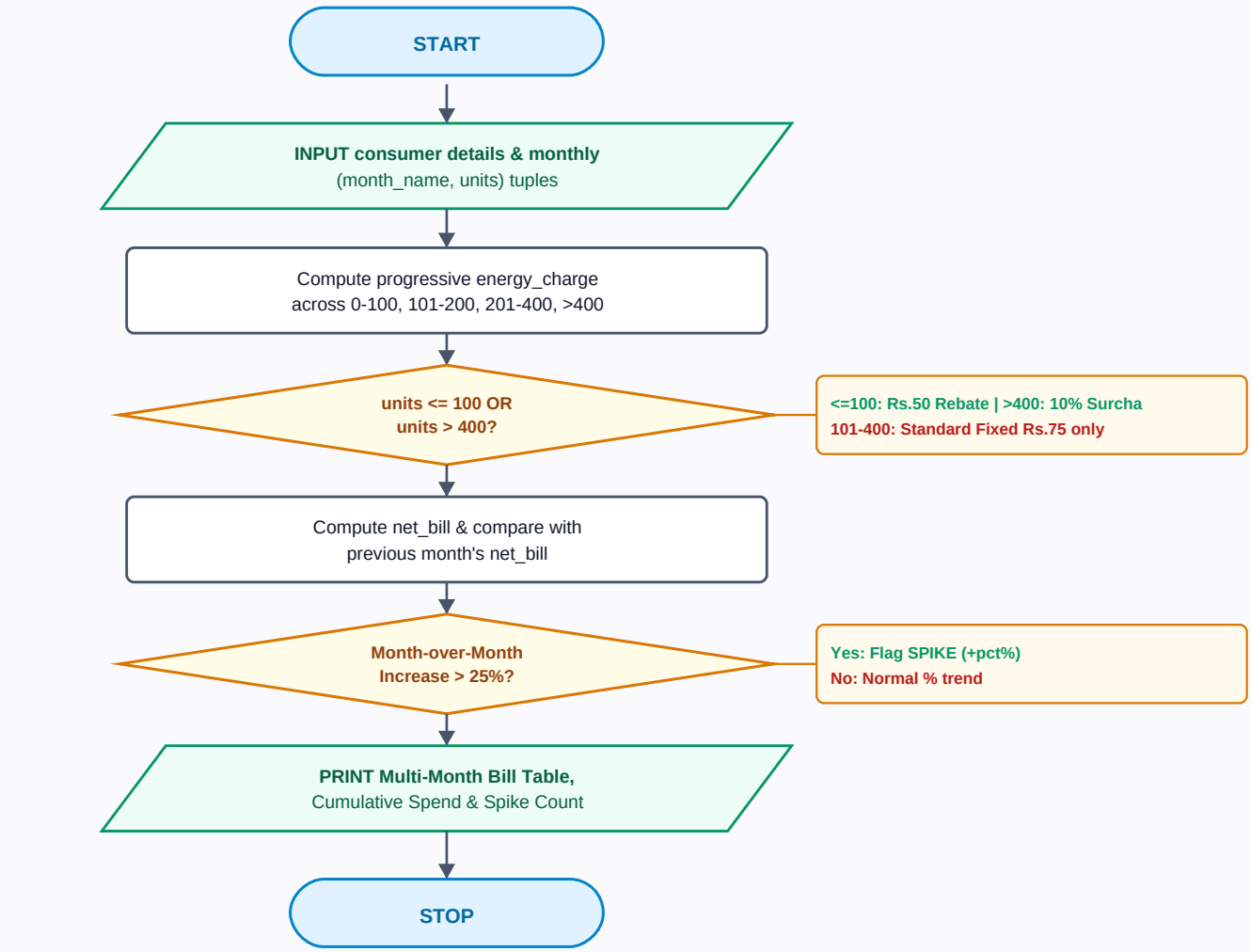
Step 7: Traverse computed_bills by index idx: for idx == 0 mark BASELINE; for idx > 0 compute pct_change = ((net - prev_net) / prev_net) * 100.

Step 8: If pct_change > 25.0, mark alert as SPIKE and record in spike_months; print formatted monthly statement and cumulative totals.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 02: Progressive Electricity Billing

```
BEGIN
    INPUT consumer_id, consumer_name, num_months
    SET fixed_charge = 75.0, computed_bills = []
    FOR i = 1 TO num_months DO
        INPUT month_label, units
        IF units <= 100 THEN energy = units * 3.0
        ELSE IF units <= 200 THEN energy = 300 + (units - 100) * 5.0
        ELSE IF units <= 400 THEN energy = 800 + (units - 200) * 7.0
        ELSE energy = 2200 + (units - 400) * 9.5
        END IF

        SET rebate = 50.0 IF units <= 100 ELSE 0.0
        SET surcharge = 0.10 * (energy + fixed_charge) IF units > 400 ELSE 0.0
        SET net_bill = energy + fixed_charge - rebate + surcharge
        APPEND (month_label, units, energy, rebate, surcharge, net_bill) TO computed_bills
    END FOR

    FOR idx = 0 TO LENGTH(computed_bills) - 1 DO
        IF idx > 0 THEN
            COMPUTE pct_change FROM computed_bills[idx - 1]
```

PSEUDOCODE — Case Study 02: Progressive Electricity Billing (continued — part 2)

```
            IF pct_change > 25.0 THEN FLAG "SPIKE" END IF
        END IF
        PRINT monthly row
    END FOR
END
```

7. Python Source Code (case_study_02.py)

case_study_02.py — Strictly Syllabus-Compliant Python 3

```python
# Case Study 2: Progressive Electricity Billing
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Lists, Tuples, Unpacking

consumer_id = input("Enter Consumer ID: ").strip()
consumer_name = input("Enter Consumer Name: ").strip()
num_months = int(input("Enter number of months: "))

monthly_records = []
for i in range(num_months):
    month_label = input("Month Name: ").strip()
    units = float(input("Units Consumed: "))
    monthly_records.append((month_label, units))

fixed_charge = 75.0
computed_bills = []

for record in monthly_records:
    month_label, units = record
    if units < 0:
        units = 0.0

    # Progressive slab computation
    if units <= 100:
        energy_charge = units * 3.0
    elif units <= 200:
        energy_charge = (100 * 3.0) + ((units - 100) * 5.0)
    elif units <= 400:
        energy_charge = (100 * 3.0) + (100 * 5.0) + ((units - 200) * 7.0)
    else:
        energy_charge = (100 * 3.0) + (100 * 5.0) + (200 * 7.0) + ((units - 400) * 9.5)

    rebate = 0.0
    surcharge = 0.0
    if units <= 100:
        rebate = 50.0
        if energy_charge < rebate:
            rebate = energy_charge
    elif units > 400:
        surcharge = 0.10 * (energy_charge + fixed_charge)

    net_bill = energy_charge + fixed_charge - rebate + surcharge
    computed_bills.append((month_label, units, energy_charge, rebate, surcharge, net_bill))

print("\n" + "=" * 72)
print(f"CONSUMER: {consumer_id} - {consumer_name} | PROGRESSIVE ELECTRICITY STATEMENT")
print("=" * 72)
print(f"{'Month':<10}{'Units':<9}{'Energy':<10}{'Rebate':<9}{'Surchg':<9}{'Total(Rs)':<11}{'Trend Alert'}")
print("-" * 72)

total_annual = 0.0
```

case_study_02.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
spike_months = []

for idx in range(len(computed_bills)):
    m_name, u_val, e_chg, reb, sur, net = computed_bills[idx]
    total_annual = total_annual + net
    if idx == 0:
        alert = "BASELINE"
    else:
        prev_net = computed_bills[idx - 1][5]
        if prev_net > 0:
            pct_change = ((net - prev_net) / prev_net) * 100.0
        else:
            pct_change = 0.0
        if pct_change > 25.0:
            alert = f"SPIKE (+{pct_change:.1f}%)"
            spike_months.append((m_name, pct_change))
        else:
            alert = f"{pct_change:+.1f}%"
    print(f"{m_name:<10}{u_val:<9.1f}{e_chg:<10.2f}{reb:<9.2f}{sur:<9.2f}{net:<11.2f}{
    alert}")

print("-" * 72)
print(f"Cumulative Billing: Rs. {total_annual:.2f} | Unusual Spikes Detected: {len(spi
    ke_months)}")
print("=" * 72)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-02A	Rebate & Multi-Slab Spike	Jan: 85u, Feb: 160u, Mar: 210u, Apr: 450u	Jan Rs.280 (Rebate 50), Feb Rs.675 (SPIKE), Apr Rs.3025 (Surcharge + SPIKE)	[PASS] Jan 280.00, Feb 675.00 (+141.1%), Apr 3025.00 (+220.1%)
TC-02B	Steady Low Usage	Jun: 95u, Jul: 98u, Aug: 100u	Rs.50 rebate applied all 3 months; 0 spikes detected	[PASS] Bills 310.00, 319.00, 325.00 | Spikes: 0

Test Case 1: All 4 Slabs, Low-Usage Rebate, Surcharge & Spike Detection — Input Session Screenshot

4 months (85, 160, 210, 450 units) exercising low-consumption rebate (Jan), normal slab progression, and >400-unit 10% surcharge with >25% spike alerts.

Terminal — Test Case 1 Input Screenshot (python3 case_study_02.py)

```
$ python3 case_study_02.py
Enter Consumer ID: C-4092
Enter Consumer Name: Rajesh Sharma
Enter number of months: 4
Month Name: Jan
Units Consumed: 85
Month Name: Feb
Units Consumed: 160
Month Name: Mar
Units Consumed: 210
Month Name: Apr
Units Consumed: 450
```

Test Case 2: Steady Low-Consumption Household — Input Session Screenshot

3 months (95, 98, 100 units) consistently within Slab 1 (<=100 units), earning Rs. 50 rebate every month with zero unusual spikes.

Terminal — Test Case 2 Input Screenshot (python3 case_study_02.py)

```
$ python3 case_study_02.py
Enter Consumer ID: C-5110
Enter Consumer Name: Priya Nair
Enter number of months: 3
Month Name: Jun
```

Terminal — Test Case 2 Input Screenshot (python3 case_study_02.py) (continued — part 2)

```
Units Consumed: 95
Month Name: Jul
Units Consumed: 98
Month Name: Aug
Units Consumed: 100
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_02.py)

```
======================================================================
CONSUMER: C-4092 - Rajesh Sharma | PROGRESSIVE ELECTRICITY STATEMENT
======================================================================
Month     Units    Energy    Rebate   Surchg   Total(Rs)  Trend Alert
----------------------------------------------------------------------
Jan       85.0     255.00    50.00    0.00     280.00     BASELINE
Feb       160.0    600.00    0.00     0.00     675.00     SPIKE (+141.1%)
Mar       210.0    870.00    0.00     0.00     945.00     SPIKE (+40.0%)
Apr       450.0    2675.00   0.00     275.00   3025.00    SPIKE (+220.1%)
----------------------------------------------------------------------
Cumulative Billing: Rs. 4925.00 | Unusual Spikes Detected: 3
======================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_02.py)

```
======================================================================
CONSUMER: C-5110 - Priya Nair | PROGRESSIVE ELECTRICITY STATEMENT
======================================================================
Month     Units    Energy    Rebate   Surchg   Total(Rs)  Trend Alert
----------------------------------------------------------------------
Jun       95.0     285.00    50.00    0.00     310.00     BASELINE
Jul       98.0     294.00    50.00    0.00     319.00     +2.9%
Aug       100.0    300.00    50.00    0.00     325.00     +1.9%
----------------------------------------------------------------------
Cumulative Billing: Rs. 954.00 | Unusual Spikes Detected: 0
======================================================================
```

10. Conclusion

This solution models piecewise linear tariff slabs, conservation incentives, high-load surcharges, and sequential anomaly detection across monthly tuple records using conditional statements and indexed list traversal.

CASE STUDY 03 OF 10

3. Hospital Billing and Patient Classification

1. Problem Statement

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

2. Objectives

- Validate hospital billing inputs against domain constraints (positive age and consultation charge, non-negative diagnostic/treatment/insurance values).
- Aggregate consultation, diagnostic, and treatment fees and apply hierarchical concession rules (BPL, Senior Citizen, High-Bill relief).
- Offset net hospital bills against patient insurance limits to compute out-of-pocket payable amounts.
- Classify patients into CASHLESS_COVERED, CO_PAY_REQUIRED, SELF_PAY_PRIORITY, or SELF_PAY_STANDARD categories.

3. Assumptions & Domain Rules

- Valid billing record requires age > 0, consultation > 0, diagnostic >= 0, treatment >= 0, and insurance_limit >= 0.
- Concession hierarchy (mutually exclusive highest-applicable tier): BPL category -> 35%, Age >= 60 / SENIOR -> 20%, Gross Bill > Rs. 50,000 -> 10%.
- Payment Policy Classification: Insurance >= Net Bill -> CASHLESS_COVERED; 0 < Insurance < Net Bill -> CO_PAY_REQUIRED.
- Uninsured patients (Insurance == 0) are classified as SELF_PAY_PRIORITY if Net Bill >= Rs. 25,000, else SELF_PAY_STANDARD.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read num_patients and collect patient tuples (pid, pname, age, category, consult, diag, treat, ins_limit).

Step 3: Initialize invalid_records = [] and print report table header.

Step 4: Unpack each patient tuple and check validity: if age <= 0 or consult <= 0 or diag < 0 or treat < 0 or ins_limit < 0, append to invalid_records and continue.

Step 5: Compute gross = consult + diag + treat.

Step 6: Determine concession: 35% if BPL, else 20% if age >= 60 or SENIOR, else 10% if gross > 50000, else 0. Compute net_bill = gross - concession.

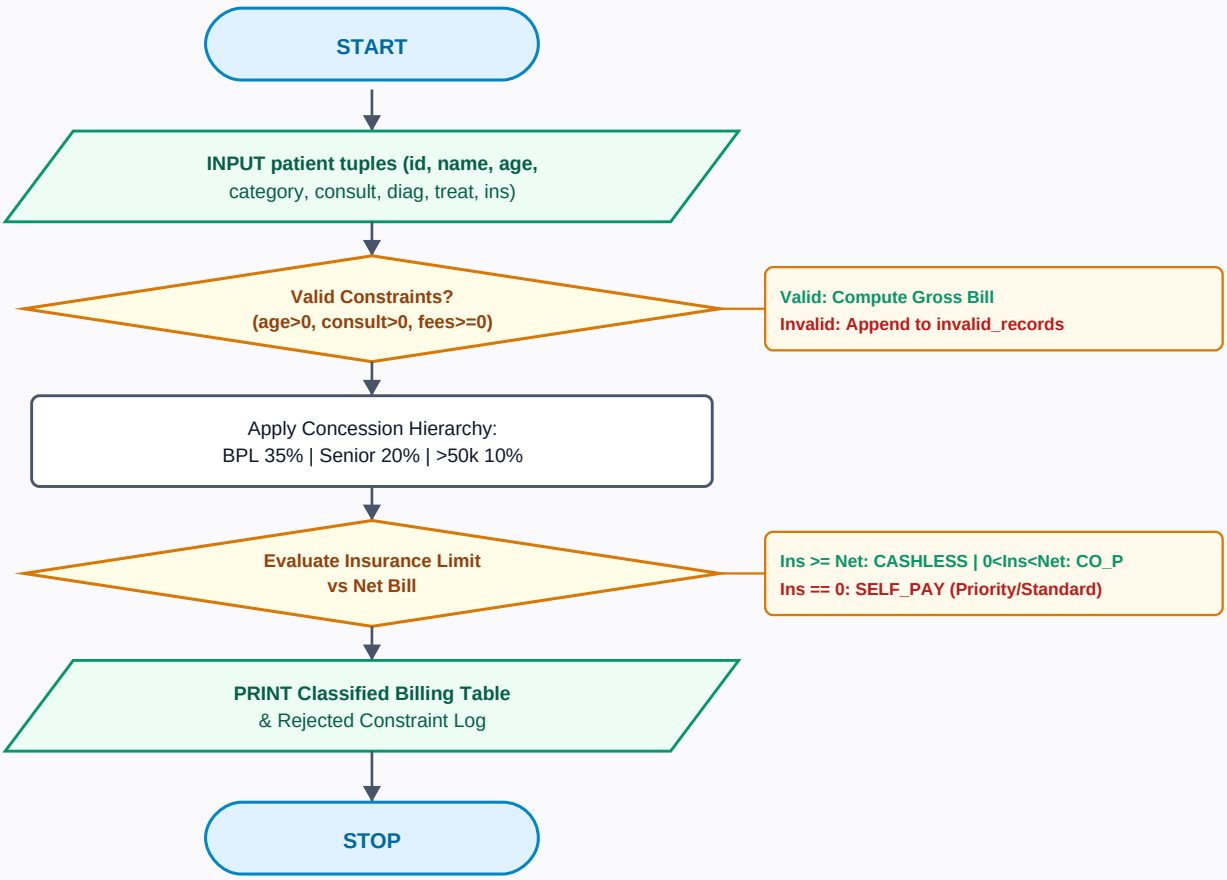
Step 7: Compare ins_limit with net_bill to compute patient_due and assign policy_class (CASHLESS_COVERED, CO_PAY_REQUIRED, SELF_PAY_PRIORITY, or SELF_PAY_STANDARD).

Step 8: Print classified patient row, and after the loop display any rejected records from invalid_records.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 03: Hospital Billing and Patient Classification

```
BEGIN
    INPUT num_patients
    READ patient tuples INTO patients
    SET invalid_records = []

    FOR EACH (pid, pname, age, category, consult, diag, treat, ins_limit) IN patients DO
        IF age <= 0 OR consult <= 0 OR diag < 0 OR treat < 0 OR ins_limit < 0 THEN
            APPEND (pid, pname, "Constraint violation") TO invalid_records
        ELSE
            SET gross = consult + diag + treat
            IF category == "BPL" THEN concession = 0.35 * gross
            ELSE IF age >= 60 OR category == "SENIOR" THEN concession = 0.20 * gross
            ELSE IF gross > 50000 THEN concession = 0.10 * gross
            ELSE concession = 0.0
            END IF

            SET net_bill = gross - concession
            CLASSIFY policy_class AND patient_due USING ins_limit AND net_bill
            PRINT pid, pname, gross, concession, net_bill, patient_due, policy_class
        END IF
    END FOR
    PRINT invalid_records IF NOT EMPTY
END
```

7. Python Source Code (case_study_03.py)

case_study_03.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 3: Hospital Billing and Patient Classification
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Lists, Tuples, Unpacking

num_patients = int(input("Enter number of patients: "))
patients = []

for i in range(num_patients):
    pid = input("Patient ID: ").strip()
    pname = input("Patient Name: ").strip()
    age = int(input("Age: "))
    category = input("Category (GENERAL/SENIOR/BPL): ").strip().upper()
    consult = float(input("Consultation Charge: "))
    diag = float(input("Diagnostic Charge: "))
    treat = float(input("Treatment Charge: "))
    ins_limit = float(input("Insurance Coverage Limit: "))
    patients.append((pid, pname, age, category, consult, diag, treat, ins_limit))

print("\n" + "=" * 74)
print("          HOSPITAL BILLING AND PATIENT CLASSIFICATION REPORT")
print("=" * 74)
print(f"{'ID':<7}{'Name':<12}{'Gross(Rs)':<11}{'Conc(Rs)':<10}{'Net(Rs)':<11}{'Payable':<10}{'Policy Class'}")
print("-" * 74)

invalid_records = []

for p in patients:
    pid, pname, age, category, consult, diag, treat, ins_limit = p

    # Validate billing constraints
    if age <= 0 or consult <= 0 or diag < 0 or treat < 0 or ins_limit < 0:
        invalid_records.append((pid, pname, "Constraint violation: non-positive age/consult or negative fee"))
    else:
        gross = consult + diag + treat
        if category == "BPL":
            concession = 0.35 * gross
        elif age >= 60 or category == "SENIOR":
            concession = 0.20 * gross
        elif gross > 50000.0:
            concession = 0.10 * gross
        else:
            concession = 0.0

        net_bill = gross - concession

        if ins_limit >= net_bill and net_bill > 0:
            patient_due = 0.0
            policy_class = "CASHLESS_COVERED"
        elif ins_limit > 0 and ins_limit < net_bill:
            patient_due = net_bill - ins_limit
            policy_class = "CO_PAY_REQUIRED"
        elif net_bill >= 25000.0:
            patient_due = net_bill
            policy_class = "SELF_PAY_PRIORITY"
        else:
            patient_due = net_bill
            policy_class = "SELF_PAY_STANDARD"

        print(f"{pid:<7}{pname:<12}{gross:<11.2f}{concession:<10.2f}{net_bill:<11.2f}{patient_due:<10.2f}{policy_class}")

if len(invalid_records) > 0:
```

case_study_03.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
61      print("-" * 74)
62      print("REJECTED BILLING RECORDS:")
63      for inv in invalid_records:
64          inv_id, inv_name, inv_msg = inv
65          print(f"{inv_id:<7}{inv_name:<12} -> {inv_msg}")
66  print("=" * 74)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-03A	Cashless Senior & Co-Pay BPL	P01 Senior (Gross 20k, Ins 20k); P02 BPL (Gross 30k, Ins 12k)	P01 Net 16k -> CASHLESS_COVERED; P02 Net 19.5k -> CO_PAY (Due 7.5k)	[PASS] P01 Due 0.00 CASHLESS | P02 Due 7500.00 CO_PAY_REQUIRED
TC-03B	Constraint Violation Guard	P04 InvalidRec: Age -5, Consult 0	Record rejected and logged under REJECTED BILLING RECORDS	[PASS] P04 rejected with constraint violation message

Test Case 1: Senior, BPL, High-Bill Concessions & Constraint Validation — Input Session Screenshot

4 records covering Cashless Senior (P01), Co-Pay BPL (P02), Self-Pay Priority >50k (P03), and an invalid negative-age/zero-consult record (P04).

Terminal — Test Case 1 Input Screenshot (python3 case_study_03.py)

```
$ python3 case_study_03.py
Enter number of patients: 4
Patient ID: P01
Patient Name: Suresh
Age: 65
Category (GENERAL/SENIOR/BPL): SENIOR
Consultation Charge: 800
Diagnostic Charge: 3200
Treatment Charge: 16000
Insurance Coverage Limit: 20000
Patient ID: P02
Patient Name: Lakshmi
Age: 42
Category (GENERAL/SENIOR/BPL): BPL
Consultation Charge: 500
Diagnostic Charge: 2500
Treatment Charge: 27000
Insurance Coverage Limit: 12000
Patient ID: P03
Patient Name: Karan
Age: 34
Category (GENERAL/SENIOR/BPL): GENERAL
Consultation Charge: 1000
Diagnostic Charge: 6000
Treatment Charge: 53000
Insurance Coverage Limit: 0
Patient ID: P04
Patient Name: InvalidRec
Age: -5
Category (GENERAL/SENIOR/BPL): GENERAL
Consultation Charge: 0
Diagnostic Charge: 500
Treatment Charge: 1000
Insurance Coverage Limit: 0
```

Test Case 2: Standard Outpatient vs Partial Senior Insurance — Input Session Screenshot

2 valid patients testing SELF_PAY_STANDARD (<25,000 with zero insurance) and Senior Citizen CO_PAY_REQUIRED.

Terminal — Test Case 2 Input Screenshot (python3 case_study_03.py)

```
$ python3 case_study_03.py
Enter number of patients: 2
Patient ID: P05
Patient Name: Divya
Age: 29
Category (GENERAL/SENIOR/BPL): GENERAL
Consultation Charge: 600
Diagnostic Charge: 1400
Treatment Charge: 8000
Insurance Coverage Limit: 0
Patient ID: P06
Patient Name: Mohan
Age: 70
Category (GENERAL/SENIOR/BPL): SENIOR
Consultation Charge: 1000
Diagnostic Charge: 4000
Treatment Charge: 15000
Insurance Coverage Limit: 10000
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_03.py)

```
========================================================================
          HOSPITAL BILLING AND PATIENT CLASSIFICATION REPORT
========================================================================
ID     Name        Gross(Rs)  Conc(Rs)  Net(Rs)    Payable   Policy Class
------------------------------------------------------------------------
P01    Suresh      20000.00   4000.00   16000.00   0.00      CASHLESS_COVERED
P02    Lakshmi     30000.00   10500.00  19500.00   7500.00   CO_PAY_REQUIRED
P03    Karan       60000.00   6000.00   54000.00   54000.00  SELF_PAY_PRIORITY
------------------------------------------------------------------------
REJECTED BILLING RECORDS:
P04    InvalidRec   -> Constraint violation: non-positive age/consult or negative fee
========================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_03.py)

```
========================================================================
          HOSPITAL BILLING AND PATIENT CLASSIFICATION REPORT
========================================================================
ID     Name        Gross(Rs)  Conc(Rs)  Net(Rs)    Payable   Policy Class
------------------------------------------------------------------------
P05    Divya       10000.00   0.00      10000.00   10000.00  SELF_PAY_STANDARD
P06    Mohan       20000.00   4000.00   16000.00   6000.00   CO_PAY_REQUIRED
========================================================================
```

10. Conclusion

The Hospital Billing system enforces strict defensive input validation prior to computing multi-tier welfare/age concessions and insurance settlement classifications using structured conditionals and tuple unpacking.

CASE STUDY 04 OF 10

4. Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives; apply progressive deductions and generate a formatted salary statement.

2. Objectives

- Compute gross monthly earnings from Basic Pay, HRA (20%), DA (12%), Overtime Pay, and Rating-based Performance Incentives.
- Validate and clamp overtime hours within statutory monthly bounds (0 to 60 hours).
- Calculate statutory deductions including Provident Fund (12% of Basic), conditional Professional Tax, and 3-slab progressive Income Tax.
- Generate an aligned multi-employee payroll statement with total organizational disbursement.

3. Assumptions & Domain Rules

- Allowances: HRA = 20% of Basic Pay; DA = 12% of Basic Pay; Overtime = Rs. 350/hour (clamped to [0, 60] hours).
- Performance Incentive on Basic Pay: Rating 5 -> 15%, Rating 4 -> 10%, Rating 3 -> 5%, Rating 1–2 -> 0%.
- Statutory Deductions: PF = 12% of Basic Pay; Professional Tax = Rs. 200 if Gross > Rs. 25,000 else Rs. 0.
- Progressive Monthly Tax on Gross Pay: 0% up to Rs. 40,000; 10% on (Gross - 40,000) up to Rs. 80,000; Rs. 4,000 + 20% on (Gross - 80,000) above Rs. 80,000.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read num_emp and input employee tuples (emp_id, name, basic, ot_hours, rating).

Step 3: Initialize total_payout = 0.0 and print formatted salary statement header.

Step 4: For each employee tuple, unpack fields and clamp ot_hours between 0.0 and 60.0.

Step 5: Compute hra = 0.20 * basic, da = 0.12 * basic, overtime_pay = ot_hours * 350.0, and rating-based incentive.

Step 6: Compute gross = basic + hra + da + overtime_pay + incentive.

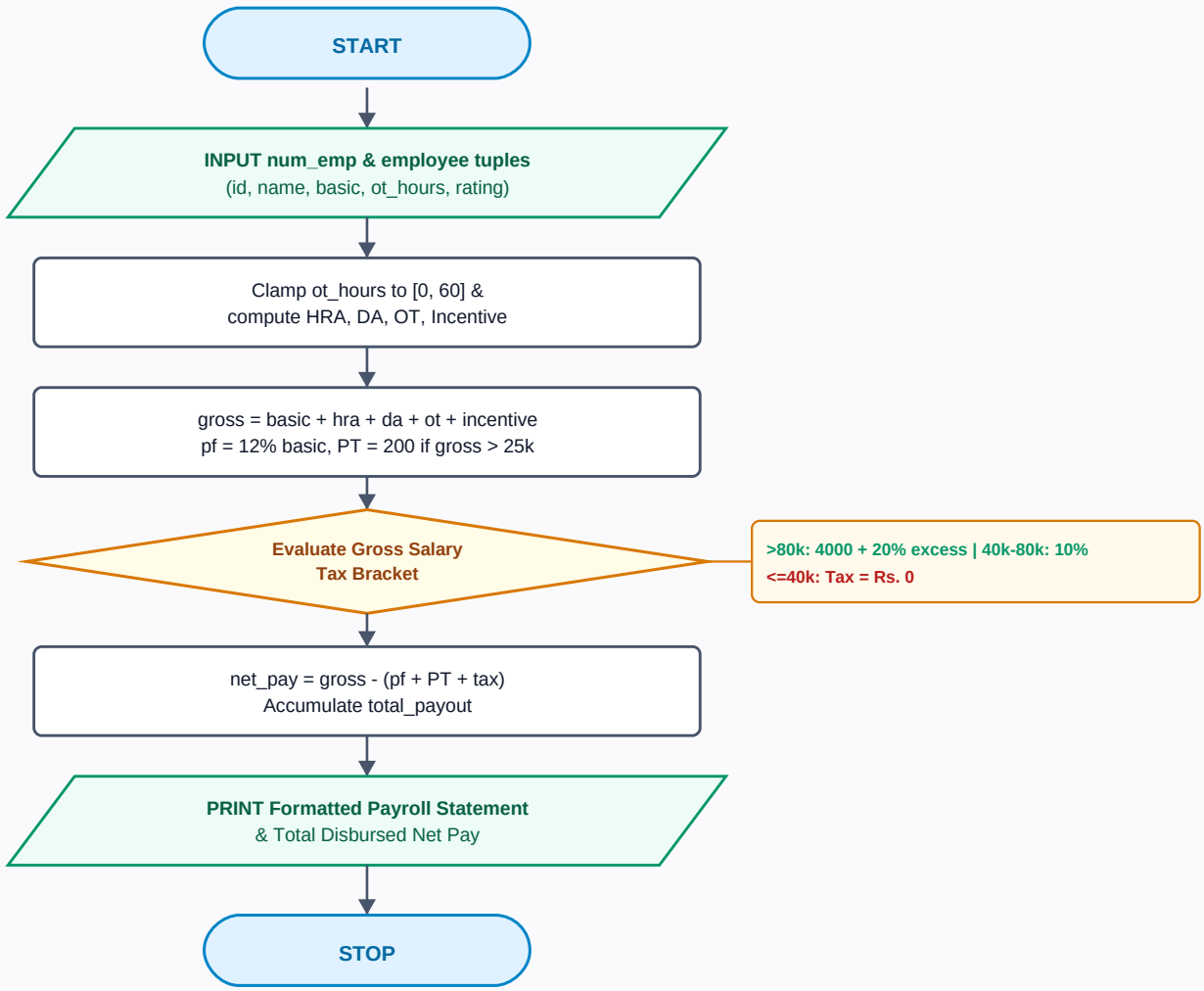
Step 7: Compute pf = 0.12 * basic, prof_tax (200 if gross > 25000 else 0), and progressive income_tax across <=40k, 40k–80k, and >80k brackets.

Step 8: Calculate net_pay = gross - (pf + prof_tax + income_tax), accumulate total_payout, and print formatted row and summary.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 04: Employee Payroll and Deduction Analysis

```
BEGIN
    INPUT num_emp
    READ employee tuples INTO employees
    SET total_payout = 0.0

    FOR EACH (emp_id, name, basic, ot_hours, rating) IN employees DO
        CLAMP ot_hours BETWEEN 0 AND 60
        SET hra = 0.20 * basic, da = 0.12 * basic, ot_pay = ot_hours * 350
        SET incentive = (15%, 10%, 5%, OR 0% OF basic BASED ON rating)
        SET gross = basic + hra + da + ot_pay + incentive

        SET pf = 0.12 * basic
        SET prof_tax = 200 IF gross > 25000 ELSE 0
        IF gross <= 40000 THEN tax = 0
        ELSE IF gross <= 80000 THEN tax = (gross - 40000) * 0.10
        ELSE tax = 4000 + (gross - 80000) * 0.20
        END IF

        SET net_pay = gross - (pf + prof_tax + tax)
        SET total_payout = total_payout + net_pay
```

PSEUDOCODE — Case Study 04: Employee Payroll and Deduction Analysis (continued — part 2)

```
        PRINT formatted employee salary row
    END FOR
    PRINT total_payout
END
```

7. Python Source Code (case_study_04.py)

case_study_04.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 4: Employee Payroll and Deduction Analysis
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Pattern Generation, Lists, Tuples

num_emp = int(input("Enter number of employees: "))
employees = []

for i in range(num_emp):
    emp_id = input("Employee ID: ").strip()
    name = input("Employee Name: ").strip()
    basic = float(input("Basic Pay (INR): "))
    ot_hours = float(input("Overtime Hours: "))
    rating = int(input("Performance Rating (1-5): "))
    employees.append((emp_id, name, basic, ot_hours, rating))

print("\n" + "=" * 74)
print("              EMPLOYEE PAYROLL & PROGRESSIVE DEDUCTION STATEMENT")
print("=" * 74)
print(f"{'ID':<7}{'Name':<12}{'Basic':<10}{'Gross':<11}{'PF+PT':<10}{'Tax':<10}{'Net Pay'}")
print("-" * 74)

total_payout = 0.0

for emp in employees:
    emp_id, name, basic, ot_hours, rating = emp
    if ot_hours < 0:
        ot_hours = 0.0
    elif ot_hours > 60.0:
        ot_hours = 60.0

    hra = 0.20 * basic
    da = 0.12 * basic
    overtime_pay = ot_hours * 350.0

    if rating == 5:
        incentive = 0.15 * basic
    elif rating == 4:
        incentive = 0.10 * basic
    elif rating == 3:
        incentive = 0.05 * basic
    else:
        incentive = 0.0

    gross = basic + hra + da + overtime_pay + incentive

    pf = 0.12 * basic
    if gross > 25000.0:
        prof_tax = 200.0
    else:
        prof_tax = 0.0

    # Progressive tax calculation on gross monthly pay
    if gross <= 40000.0:
        income_tax = 0.0
    elif gross <= 80000.0:
```

case_study_04.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
55          income_tax = (gross - 40000.0) * 0.10
56      else:
57          income_tax = 4000.0 + ((gross - 80000.0) * 0.20)
58
59      total_deductions = pf + prof_tax + income_tax
60      net_pay = gross - total_deductions
61      total_payout = total_payout + net_pay
62
63      print(f"{emp_id:<7}{name:<12}{basic:<10.2f}{gross:<11.2f}{(pf + prof_tax):<10.2f}{
        income_tax:<10.2f}{net_pay:.2f}")
64
65  print("-" * 74)
66  print(f"Total Disbursed Net Payroll: Rs. {total_payout:.2f}")
67  print("=" * 74)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-04A	Top & Mid Tax Brackets	E101 Basic 65k, OT 12h, R5; E102 Basic 38k, OT 8h, R4	E101 Gross 99,750, Tax 7,950, Net 83,800; E102 Gross 56,760, Tax 1,676	[PASS] E101 Net 83800.00 | E102 Net 50324.00
TC-04B	OT Cap & Negative Guard	E201 OT=75h (clamped to 60h); E202 OT=-4h (clamped to 0h)	E201 OT capped at 60h (Rs. 21,000); E202 OT floored at 0h	[PASS] E201 Gross 145950.00 | E202 Gross 24660.00 (PT=0)

Test Case 1: Multi-Bracket Progressive Tax & Incentive Tiers — Input Session Screenshot

3 employees across >80k (20% top slab), 40k-80k (10% middle slab), and <=40k (0% tax, exempt PT <=25k check) brackets.

Terminal — Test Case 1 Input Screenshot (python3 case_study_04.py)

```
$ python3 case_study_04.py
Enter number of employees: 3
Employee ID: E101
Employee Name: Siddharth
Basic Pay (INR): 65000
Overtime Hours: 12
Performance Rating (1-5): 5
Employee ID: E102
Employee Name: Neha
Basic Pay (INR): 38000
Overtime Hours: 8
Performance Rating (1-5): 4
Employee ID: E103
Employee Name: Rahul
Basic Pay (INR): 18000
Overtime Hours: 2
Performance Rating (1-5): 2
```

Test Case 2: Overtime Cap (>60 hrs) & Negative Overtime Guard — Input Session Screenshot

Verifies overtime clamping at 60 hours maximum for E201 and 0 hours floor for negative input (-4 hrs) on E202.

Terminal — Test Case 2 Input Screenshot (python3 case_study_04.py)

```
$ python3 case_study_04.py
Enter number of employees: 2
Employee ID: E201
Employee Name: Aditi
Basic Pay (INR): 85000
Overtime Hours: 75
Performance Rating (1-5): 5
Employee ID: E202
Employee Name: Varun
Basic Pay (INR): 18000
Overtime Hours: -4
Performance Rating (1-5): 3
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_04.py)

```
================================================================================
              EMPLOYEE PAYROLL & PROGRESSIVE DEDUCTION STATEMENT
================================================================================
ID     Name        Basic     Gross      PF+PT     Tax       Net Pay
--------------------------------------------------------------------------------
E101   Siddharth   65000.00  99750.00   8000.00   7950.00   83800.00
E102   Neha        38000.00  56760.00   4760.00   1676.00   50324.00
E103   Rahul       18000.00  24460.00   2160.00   0.00      22300.00
--------------------------------------------------------------------------------
Total Disbursed Net Payroll: Rs. 156424.00
================================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_04.py)

```
================================================================================
              EMPLOYEE PAYROLL & PROGRESSIVE DEDUCTION STATEMENT
================================================================================
ID     Name        Basic     Gross      PF+PT     Tax       Net Pay
--------------------------------------------------------------------------------
E201   Aditi       85000.00  145950.00  10400.00  17190.00  118360.00
E202   Varun       18000.00  24660.00   2160.00   0.00      22500.00
--------------------------------------------------------------------------------
Total Disbursed Net Payroll: Rs. 140860.00
================================================================================
```

10. Conclusion

The Employee Payroll analyzer demonstrates accurate earnings aggregation, overtime boundary enforcement, progressive tax computation, and tabular string formatting using core Python primitives.

CASE STUDY 05 OF 10

5. Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Evaluate several mobile plans against usage, validity, and budget constraints; calculate excess charges and identify the most economical eligible plan.

2. Objectives

- Model multiple telecom tariff plans as fixed-schema tuples containing base rental, validity, bundled quotas, and unit overage rates.
- Calculate excess data, voice, and SMS charges dynamically against user usage projections plus 18% GST.
- Filter out plans violating minimum validity days or maximum user budget constraints.
- Identify the minimum-cost eligible plan, resolving exact cost ties in favor of longer plan validity.

3. Assumptions & Domain Rules

- Four standard plans evaluated: Lite-28 (Rs.199, 28d), Value-30 (Rs.299, 30d), Smart-56 (Rs.479, 56d), and Ultra-84 (Rs.719, 84d).
- Overage applies strictly when required usage exceeds bundled quota; unused quota carries no negative credit.
- Final payable cost includes 18% telecom GST: Final = (Base + Excess_Data + Excess_Voice + Excess_SMS) * 1.18.
- Eligibility requires Plan_Validity >= Min_Validity AND Final_Cost <= Budget.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read user requirements: req_data, req_mins, req_sms, min_validity, and budget.

Step 3: Initialize catalog list plans of 9-element plan tuples and set best_plan = '', best_cost = -1.0, best_validity = 0.

Step 4: Loop through each tuple in plans and unpack base_fee, val_days, quotas, and unit overage rates.

Step 5: Compute excess_data, excess_voice, and excess_sms where required usage exceeds plan quota; sum into excess_total.

Step 6: Compute final_cost = (base_fee + excess_total) * 1.18.

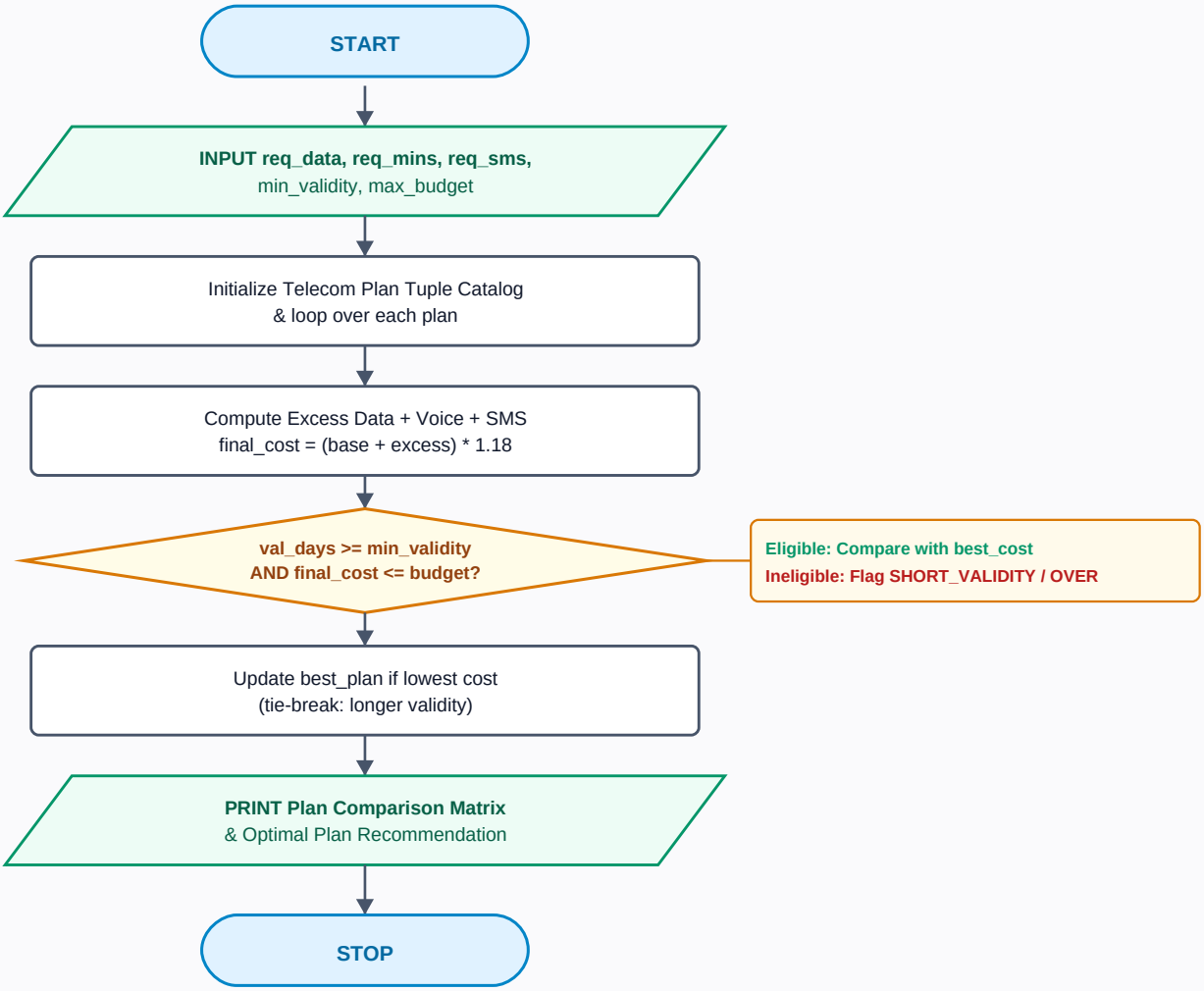
Step 7: Check constraints: if val_days < min_validity mark SHORT_VALIDITY; else if final_cost > budget mark OVER_BUDGET; else mark ELIGIBLE.

Step 8: For ELIGIBLE plans, update best_plan if best_cost < 0 or final_cost < best_cost (or equal cost with greater val_days). Print comparison table and recommendation.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 05: Multi-Plan Telecommunications Cost Optimization

```
BEGIN
    INPUT req_data, req_mins, req_sms, min_validity, budget
    INITIALIZE plans = [Lite-28, Value-30, Smart-56, Ultra-84 tuples]
    SET best_plan = "", best_cost = -1

    FOR EACH plan IN plans DO
        UNPACK (name, base, val_days, p_data, p_mins, p_sms, r_gb, r_min, r_sms) = plan
        SET ex_data = (req_data - p_data) * r_gb IF req_data > p_data ELSE 0
        SET ex_voice = (req_mins - p_mins) * r_min IF req_mins > p_mins ELSE 0
        SET ex_sms = (req_sms - p_sms) * r_sms IF req_sms > p_sms ELSE 0
        SET final_cost = (base + ex_data + ex_voice + ex_sms) * 1.18

        IF val_days >= min_validity AND final_cost <= budget THEN
            MARK "ELIGIBLE"
            IF best_cost < 0 OR final_cost < best_cost THEN
                SET best_cost = final_cost, best_plan = name
            END IF
        ELSE
            MARK "SHORT_VALIDITY" OR "OVER_BUDGET"
        END IF
```

PSEUDOCODE — Case Study 05: Multi-Plan Telecommunications Cost Optimization (continued — part 2)

```
    END FOR
    PRINT optimal recommendation OR infeasibility notice
END
```

7. Python Source Code (case_study_05.py)

case_study_05.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 5: Multi-Plan Telecommunications Cost Optimization
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Lists, Tuples, Unpacking

req_data = float(input("Required Data (GB): "))
req_mins = int(input("Required Voice Calls (Mins): "))
req_sms = int(input("Required SMS Count: "))
min_validity = int(input("Minimum Validity Needed (Days): "))
budget = float(input("Maximum Budget (INR): "))

# Plan tuple: (name, base_fee, validity, data_gb, voice_mins, sms_limit, extra_gb_rate, extra_min_rate, extra_sms_rate)
plans = [
    ("Lite-28", 199.0, 28, 15.0, 300, 100, 12.0, 0.50, 1.0),
    ("Value-30", 299.0, 30, 45.0, 1000, 300, 10.0, 0.40, 0.8),
    ("Smart-56", 479.0, 56, 84.0, 3000, 600, 8.0, 0.30, 0.5),
    ("Ultra-84", 719.0, 84, 168.0, 5000, 1000, 6.0, 0.25, 0.5)
]

print("\n" + "=" * 74)
print("          MULTI-PLAN TELECOMMUNICATIONS OPTIMIZATION REPORT")
print("=" * 74)
print(f"{'Plan':<11}{'Valid':<7}{'Base(Rs)':<10}{'Excess(Rs)':<12}{'Final+GST':<12}{'Status'}")
print("-" * 74)

best_plan = ""
best_cost = -1.0
best_validity = 0

for plan in plans:
    p_name, base_fee, val_days, p_data, p_mins, p_sms, r_gb, r_min, r_sms = plan

    excess_data = 0.0
    if req_data > p_data:
        excess_data = (req_data - p_data) * r_gb

    excess_voice = 0.0
    if req_mins > p_mins:
        excess_voice = (req_mins - p_mins) * r_min

    excess_sms = 0.0
    if req_sms > p_sms:
        excess_sms = (req_sms - p_sms) * r_sms

    excess_total = excess_data + excess_voice + excess_sms
    final_cost = (base_fee + excess_total) * 1.18

    if val_days < min_validity:
        status = "SHORT_VALIDITY"
    elif final_cost > budget:
        status = "OVER_BUDGET"
    else:
        status = "ELIGIBLE"
        if best_cost < 0 or final_cost < best_cost or (final_cost == best_cost and val_days > best_validity):
            best_cost = final_cost
```

case_study_05.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
            best_plan = p_name
            best_validity = val_days

        print(f"{p_name:<11}{val_days:<7}{base_fee:<10.2f}{excess_total:<12.2f}{final_cost:<12.2f}{status}")

print("-" * 74)
if best_plan != "":
    print(f"RECOMMENDED ECONOMICAL PLAN: {best_plan} at Rs. {best_cost:.2f} ({best_validity} days)")
else:
    print("RECOMMENDATION: No eligible plan satisfies both validity and budget constraints.")
print("=" * 74)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-05A	Overage vs Higher Base Plan	50 GB, 1200 mins, 250 SMS, >=28 days, Budget Rs.650	Value-30 (Rs.506.22 with Rs.130 excess) beats Smart-56 (Rs.565.22)	[PASS] Recommended: Value-30 at Rs. 506.22 (30 days)
TC-05B	Infeasible Validity + Budget	120 GB, 3500 mins, 400 SMS, >=60 days, Budget Rs.600	3 plans fail 60d validity; Ultra-84 (Rs.848.42) exceeds Rs.600 budget	[PASS] No eligible plan satisfies both constraints

Test Case 1: Excess Usage Trade-Off & Optimal Plan Selection — Input Session Screenshot

50 GB, 1200 mins, 250 SMS, min 28 days, Rs. 650 budget. Compares Value-30 + excess charges vs Smart-56 zero-excess base plan.

Terminal — Test Case 1 Input Screenshot (python3 case_study_05.py)

```
$ python3 case_study_05.py
Required Data (GB): 50
Required Voice Calls (Mins): 1200
Required SMS Count: 250
Minimum Validity Needed (Days): 28
Maximum Budget (INR): 650
```

Test Case 2: Strict Validity & Tight Budget Infeasibility — Input Session Screenshot

120 GB, 3500 mins, 400 SMS, min 60 days validity, Rs. 600 budget. Short-validity plans rejected and Ultra-84 exceeds budget.

Terminal — Test Case 2 Input Screenshot (python3 case_study_05.py)

```
$ python3 case_study_05.py
Required Data (GB): 120
Required Voice Calls (Mins): 3500
Required SMS Count: 400
Minimum Validity Needed (Days): 60
Maximum Budget (INR): 600
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_05.py)

```
==========================================================================
         MULTI-PLAN TELECOMMUNICATIONS OPTIMIZATION REPORT
==========================================================================
Plan       Valid  Base(Rs)  Excess(Rs)  Final+GST   Status
--------------------------------------------------------------------------
Lite-28    28     199.00    1020.00     1438.42     OVER_BUDGET
Value-30   30     299.00    130.00      506.22      ELIGIBLE
Smart-56   56     479.00    0.00        565.22      ELIGIBLE
Ultra-84   84     719.00    0.00        848.42      OVER_BUDGET
--------------------------------------------------------------------------
RECOMMENDED ECONOMICAL PLAN: Value-30 at Rs. 506.22 (30 days)
```

Terminal — Test Case 1 Verified Output Screenshot (case_study_05.py) (continued — part 2)

```
==========================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_05.py)

```
==========================================================================
         MULTI-PLAN TELECOMMUNICATIONS OPTIMIZATION REPORT
==========================================================================
Plan       Valid  Base(Rs)  Excess(Rs)  Final+GST   Status
--------------------------------------------------------------------------
Lite-28    28     199.00    3160.00     3963.62     SHORT_VALIDITY
Value-30   30     299.00    1830.00     2512.22     SHORT_VALIDITY
Smart-56   56     479.00    438.00      1082.06     SHORT_VALIDITY
Ultra-84   84     719.00    0.00        848.42      OVER_BUDGET
--------------------------------------------------------------------------
RECOMMENDATION: No eligible plan satisfies both validity and budget constraints.
==========================================================================
```

10. Conclusion

The optimizer evaluates fixed vs. variable telecom overage trade-offs across tuple catalogs, demonstrating linear scan minimization under compound validity and budget constraints.

CASE STUDY 06 OF 10

6. Railway Fare and Reservation Calculator

1. Problem Statement

Calculate fares for multiple passengers using distance slabs, travel classes, age-based concessions, reservation fees, and specified cancellation penalties.

2. Objectives

- Compute telescopic railway distance fares across 0–100 km, 101–300 km, and >300 km slabs.
- Scale base fares using travel class multipliers and fixed reservation charges unpacked from class tuples (SL, 3A, 2A).
- Apply age-based passenger concessions for Infants (<5 yrs free), Children (5–11 yrs 50% off), and Senior Citizens (>=60 yrs 40% off).
- Process confirmed vs. cancelled booking statuses (24-hour vs. last-minute cancellation penalties) across multi-passenger PNR lists.

3. Assumptions & Domain Rules

- Telescopic Sleeper (SL) distance rate: first 100 km @ Rs. 1.20/km, next 200 km @ Rs. 1.00/km, >300 km @ Rs. 0.85/km.
- Class (Multiplier, Reservation Fee): SL = (1.0x, Rs. 20), 3A = (2.2x, Rs. 40), 2A = (3.2x, Rs. 50).
- Age Concessions: Age < 5 -> 100% Free (no res fee); 5 <= Age < 12 -> 50% off fare; Age >= 60 -> 40% off fare.
- Cancellation Penalties: CANCEL_24H retains 25% of discounted fare + reservation fee; CANCEL_LAST_MIN retains 50% + reservation fee.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read journey distance, travel_class (SL/3A/2A), and num_passengers; store (pname, age, status) tuples in passengers.

Step 3: Compute telescopic slab_fare for distance across <=100, 101–300, and >300 km.

Step 4: Assign (class_mult, res_fee) tuple based on travel_class and unpack it.

Step 5: Loop over each passenger tuple (pname, age, status) and calculate base_class_fare = slab_fare * class_mult.

Step 6: Apply age concession: if age < 5 set ticket_fare = 0; else apply 50% (child), 40% (senior), or 0% (adult) discount and add res_fee.

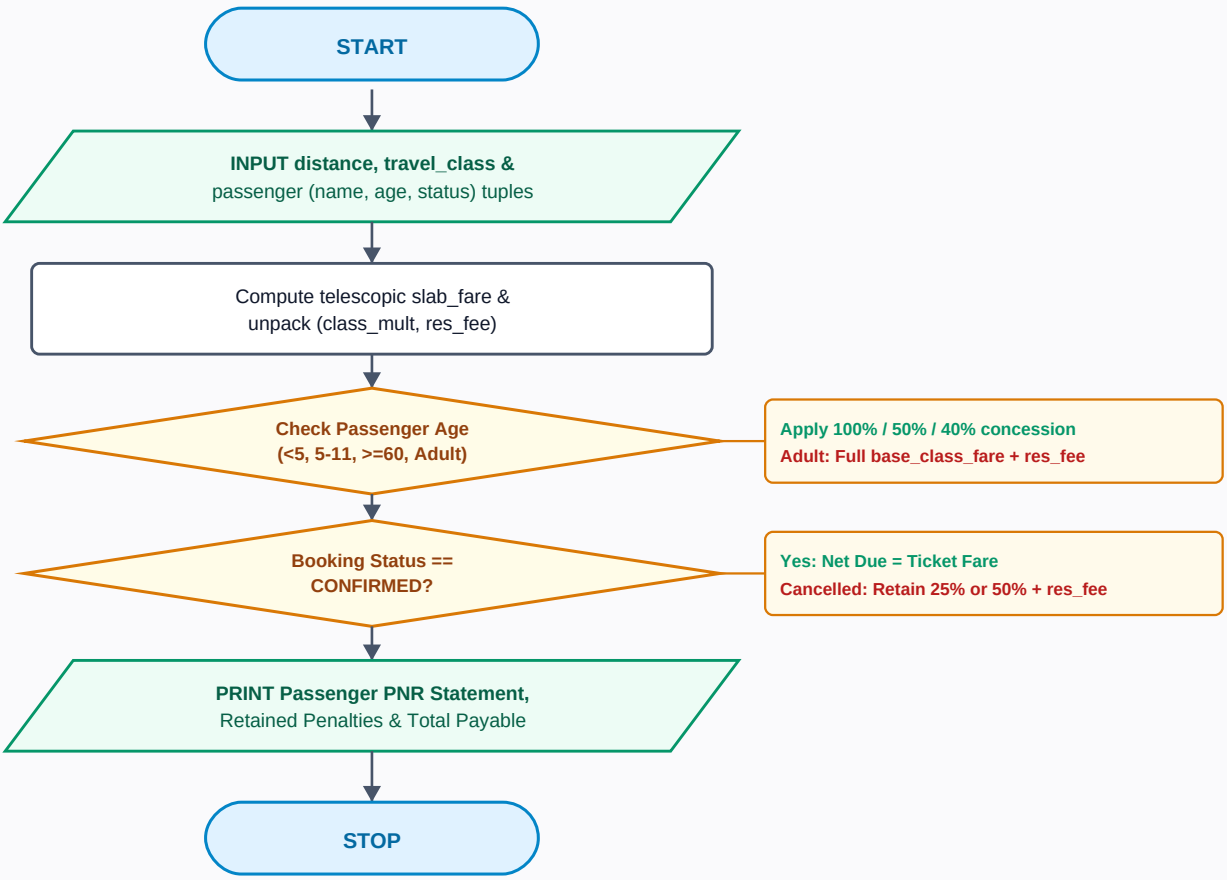
Step 7: Evaluate booking status: if CONFIRMED, net_due = ticket_fare; if CANCEL_24H, net_due = 25% disc_fare + res_fee; if CANCEL_LAST_MIN, net_due = 50% disc_fare + res_fee.

Step 8: Accumulate total_penalties and total_payable, print passenger breakdown and PNR summary.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 06: Railway Fare and Reservation Calculator

```
BEGIN
    INPUT distance, travel_class, num_passengers
    READ (pname, age, status) tuples INTO passengers
    COMPUTE slab_fare FROM distance USING (1.20, 1.00, 0.85) slabs
    UNPACK (class_mult, res_fee) BASED ON travel_class

    SET total_payable = 0, total_penalties = 0
    FOR EACH (pname, age, status) IN passengers DO
        SET base_fare = slab_fare * class_mult
        IF age < 5 THEN disc_fare = 0, applied_res = 0
        ELSE IF age < 12 THEN disc_fare = 0.50 * base_fare, applied_res = res_fee
        ELSE IF age >= 60 THEN disc_fare = 0.60 * base_fare, applied_res = res_fee
        ELSE disc_fare = base_fare, applied_res = res_fee
        END IF

        IF status == "CANCEL_24H" THEN net_due = 0.25 * disc_fare + applied_res
        ELSE IF status == "CANCEL_LAST_MIN" THEN net_due = 0.50 * disc_fare + applied_res
        ELSE net_due = disc_fare + applied_res
        END IF
        ACCUMULATE totals AND PRINT passenger row
    END FOR
END
```

7. Python Source Code (case_study_06.py)

case_study_06.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 6: Railway Fare and Reservation Calculator
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Lists, Tuples, Unpacking

distance = float(input("Enter Journey Distance (km): "))
travel_class = input("Enter Travel Class (SL/3A/2A): ").strip().upper()
num_passengers = int(input("Enter Number of Passengers: "))

passengers = []
for i in range(num_passengers):
    pname = input("Passenger Name: ").strip()
    age = int(input("Age: "))
    status = input("Booking Status (CONFIRMED/CANCEL_24H/CANCEL_LAST_MIN): ").strip().upper()
    passengers.append((pname, age, status))

# Distance slab base fare calculation (Sleeper baseline)
if distance <= 100.0:
    slab_fare = distance * 1.20
elif distance <= 300.0:
    slab_fare = (100.0 * 1.20) + ((distance - 100.0) * 1.00)
else:
    slab_fare = (100.0 * 1.20) + (200.0 * 1.00) + ((distance - 300.0) * 0.85)

# Class multiplier and reservation fee tuple unpacking
if travel_class == "2A":
    class_info = (3.2, 50.0)
elif travel_class == "3A":
    class_info = (2.2, 40.0)
else:
    travel_class = "SL"
    class_info = (1.0, 20.0)

class_mult, res_fee = class_info

print("\n" + "=" * 76)
print(f"RAILWAY FARE & RESERVATION SUMMARY | DIST: {distance:.0f} km | CLASS: {travel_class}")
print("=" * 76)
print(f"{'Passenger':<13}{'Age':<6}{'Status':<17}{'Ticket(Rs)':<13}{'Penalty':<11}{'Net Due'}")
print("-" * 76)

total_payable = 0.0
total_penalties = 0.0

for p in passengers:
    pname, age, status = p
    base_class_fare = slab_fare * class_mult

    if age < 5:
        ticket_fare = 0.0
        disc_fare = 0.0
        applied_res = 0.0
    elif age < 12:
        disc_fare = base_class_fare * 0.50
        applied_res = res_fee
        ticket_fare = disc_fare + applied_res
    elif age >= 60:
        disc_fare = base_class_fare * 0.60
        applied_res = res_fee
        ticket_fare = disc_fare + applied_res
    else:
        disc_fare = base_class_fare
```

case_study_06.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
            applied_res = res_fee
            ticket_fare = disc_fare + applied_res

        if status == "CANCEL_24H":
            penalty = (0.25 * disc_fare) + applied_res
            net_due = penalty
        elif status == "CANCEL_LAST_MIN":
            penalty = (0.50 * disc_fare) + applied_res
            net_due = penalty
        else:
            status = "CONFIRMED"
            penalty = 0.0
            net_due = ticket_fare

        if age < 5:
            penalty = 0.0
            net_due = 0.0

        total_penalties = total_penalties + penalty
        total_payable = total_payable + net_due
        print(f"{pname:<13}{age:<6}{status:<17}{ticket_fare:<13.2f}{penalty:<11.2f}{net_du
        e:.2f}")

    print("-" * 76)
    print(f"Cancellation Charges Retained: Rs. {total_penalties:.2f} | Final Amount Payabl
        e: Rs. {total_payable:.2f}")
    print("=" * 76)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-06A	4-Passenger Family Mix (3A)	420km 3A: Adult(34), Senior(64), Child(9, CANCEL_24H), Infant(3)	Adult 968.40, Senior 597.04, Child Penalty 156.05, Infant 0.00	[PASS] Penalties: Rs.156.05 | Payable: Rs.1721.49
TC-06B	2A Last-Minute Cancellation	180km 2A: Adult(28, CANCEL_LAST_MIN), Senior(62, CONFIRMED)	Adult 50% penalty + Rs.50 res = Rs.370.00; Senior = Rs.434.00	[PASS] Penalties: Rs.370.00 | Payable: Rs.804.00

Test Case 1: Mixed Family Booking (Adult, Senior, Child Cancel, Infant) — Input Session Screenshot

420 km in 3A class for 4 passengers: Adult Confirmed, Senior (40% off), Child 9 yrs (50% off + 24h cancellation penalty), Infant 3 yrs (free).

Terminal — Test Case 1 Input Screenshot (python3 case_study_06.py)

```
$ python3 case_study_06.py
Enter Journey Distance (km): 420
Enter Travel Class (SL/3A/2A): 3A
Enter Number of Passengers: 4
Passenger Name: Aravind
Age: 34
Booking Status (CONFIRMED/CANCEL_24H/CANCEL_LAST_MIN): CONFIRMED
Passenger Name: Kamala
Age: 64
Booking Status (CONFIRMED/CANCEL_24H/CANCEL_LAST_MIN): CONFIRMED
Passenger Name: Riya
Age: 9
Booking Status (CONFIRMED/CANCEL_24H/CANCEL_LAST_MIN): CANCEL_24H
Passenger Name: Aarav
Age: 3
Booking Status (CONFIRMED/CANCEL_24H/CANCEL_LAST_MIN): CONFIRMED
```

Test Case 2: 2A Class Last-Minute Cancellation Penalty — Input Session Screenshot

180 km in 2A class testing 50% last-minute cancellation deduction + reservation fee retention alongside Senior Confirmed ticket.

Terminal — Test Case 2 Input Screenshot (python3 case_study_06.py)

```
$ python3 case_study_06.py
Enter Journey Distance (km): 180
Enter Travel Class (SL/3A/2A): 2A
Enter Number of Passengers: 2
Passenger Name: Nikhil
Age: 28
Booking Status (CONFIRMED/CANCEL_24H/CANCEL_LAST_MIN): CANCEL_LAST_MIN
Passenger Name: Sneha
Age: 62
Booking Status (CONFIRMED/CANCEL_24H/CANCEL_LAST_MIN): CONFIRMED
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_06.py)

```
================================================================================
RAILWAY FARE & RESERVATION SUMMARY | DIST: 420 km | CLASS: 3A
================================================================================
Passenger    Age   Status           Ticket(Rs)   Penalty    Net Due
--------------------------------------------------------------------------------
Aravind      34    CONFIRMED        968.40       0.00       968.40
Kamala       64    CONFIRMED        597.04       0.00       597.04
Riya         9     CANCEL_24H       504.20       156.05     156.05
Aarav        3     CONFIRMED        0.00         0.00       0.00
--------------------------------------------------------------------------------
Cancellation Charges Retained: Rs. 156.05 | Final Amount Payable: Rs. 1721.49
================================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_06.py)

```
================================================================================
RAILWAY FARE & RESERVATION SUMMARY | DIST: 180 km | CLASS: 2A
================================================================================
Passenger    Age   Status           Ticket(Rs)   Penalty    Net Due
--------------------------------------------------------------------------------
Nikhil       28    CANCEL_LAST_MIN  690.00       370.00     370.00
Sneha        62    CONFIRMED        434.00       0.00       434.00
--------------------------------------------------------------------------------
Cancellation Charges Retained: Rs. 370.00 | Final Amount Payable: Rs. 804.00
================================================================================
```

10. Conclusion

The Railway Fare calculator combines telescopic distance pricing, tuple-unpacked class configurations, multi-tier age concessions, and cancellation penalty retention in a single cohesive pass.

CASE STUDY 07 OF 10

7. Retail Billing with Multiple Discount Rules

1. Problem Statement

Process multiple products, quantities, category-specific taxes, threshold discounts, and maximum concession limits to generate an itemized invoice.

2. Objectives

- Process multi-item retail baskets stored as tuples and intermediate nested lists.
- Apply category-specific bulk quantity discounts (ESSENTIAL >=5, APPAREL >=3, ELECTRONICS >=2) and GST rates (5%, 12%, 18%).
- Evaluate cart-wide threshold discounts (7% extra off when post-item-discount subtotal exceeds Rs. 10,000).
- Enforce a strict Rs. 3,000 maximum total concession cap by proportionally scaling line-item concessions prior to GST calculation.

3. Assumptions & Domain Rules

- Categories & GST: ESSENTIAL (5% GST, 5% bulk off if qty >= 5), APPAREL (12% GST, 10% off if qty >= 3), ELECTRONICS (18% GST, 8% off if qty >= 2).
- Threshold Discount: Additional 7% cart discount if subtotal after bulk item discounts exceeds Rs. 10,000.
- Maximum Concession Cap: Combined item + threshold concessions cannot exceed Rs. 3,000.00 across the entire invoice.
- GST is levied on each item's effective taxable value after capped concession allocation.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read num_items and collect product tuples (code, name, category, price, qty) into cart.

Step 3: First pass: skip non-positive qty/price; compute line_gross = price * qty, category bulk discount l_disc, and store [code, name, category, qty, line_gross, l_disc, tax_rate] in nested list line_items.

Step 4: Compute after_item_disc = gross_subtotal - item_discount_total. If after_item_disc > 10000, set threshold_disc = 0.07 * after_item_disc, else 0.

Step 5: Check concession cap: raw_total_disc = item_discount_total + threshold_disc. If raw_total_disc > 3000, set scale_factor = 3000 / raw_total_disc and cap_applied = True; else scale_factor = 1.0.

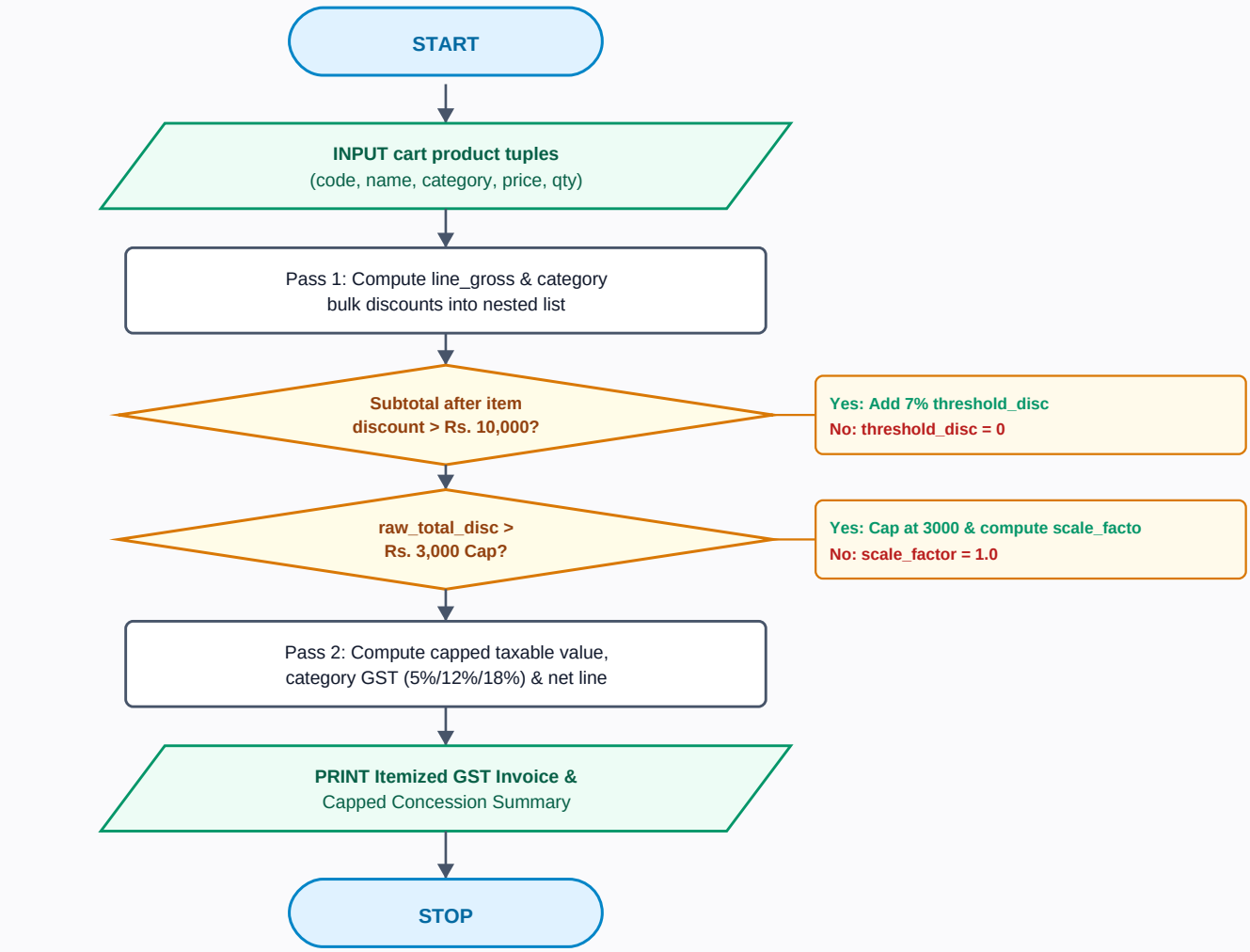
Step 6: Second pass over line_items: allocate proportional threshold share, multiply by scale_factor to get effective_disc, compute taxable_val = l_gross - effective_disc, and gst_amt = taxable_val * t_rate.

Step 7: Sum gst_amt and net_line into invoice totals and print itemized rows with concession cap notice.

Step 8: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 07: Retail Billing with Multiple Discount Rules

```
BEGIN
    INPUT num_items AND READ product tuples INTO cart
    SET line_items = [], gross_subtotal = 0, item_disc_total = 0

    FOR EACH (code, name, cat, price, qty) IN cart DO
        SET line_gross = price * qty
        DETERMINE (tax_rate, bulk_disc_rate) BY cat AND qty
        SET l_disc = line_gross * bulk_disc_rate
        APPEND [code, name, cat, qty, line_gross, l_disc, tax_rate] TO line_items
        ACCUMULATE gross_subtotal AND item_disc_total
    END FOR

    SET after_item = gross_subtotal - item_disc_total
    SET thresh_disc = 0.07 * after_item IF after_item > 10000 ELSE 0
    SET raw_disc = item_disc_total + thresh_disc
    SET scale = 3000.0 / raw_disc IF raw_disc > 3000.0 ELSE 1.0

    FOR EACH row IN line_items DO
        COMPUTE capped effective_disc USING scale
        SET taxable = l_gross - effective_disc
```

PSEUDOCODE — Case Study 07: Retail Billing with Multiple Discount Rules (continued — part 2)

```
        SET gst = taxable * tax_rate, net = taxable + gst
        PRINT itemized invoice line
    END FOR
    PRINT Subtotal, Capped Concession, Total GST, Grand Total
END
```

7. Python Source Code (case_study_07.py)

case_study_07.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 7: Retail Billing with Multiple Discount Rules
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Nested Lists, Tuples, Unpacking

num_items = int(input("Enter number of distinct products: "))
cart = []

for i in range(num_items):
    code = input("Product Code: ").strip()
    name = input("Product Name: ").strip()
    category = input("Category (ESSENTIAL/APPAREL/ELECTRONICS): ").strip().upper()
    price = float(input("Unit Price (INR): "))
    qty = int(input("Quantity: "))
    cart.append((code, name, category, price, qty))

line_items = []
gross_subtotal = 0.0
item_discount_total = 0.0

for item in cart:
    code, name, category, price, qty = item
    if qty <= 0 or price < 0:
        continue

    line_gross = price * qty
    gross_subtotal = gross_subtotal + line_gross

    # Category-specific bulk discount and GST rate tuple
    if category == "ELECTRONICS":
        tax_rate = 0.18
        disc_rate = 0.08 if qty >= 2 else 0.0
    elif category == "APPAREL":
        tax_rate = 0.12
        disc_rate = 0.10 if qty >= 3 else 0.0
    else:
        category = "ESSENTIAL"
        tax_rate = 0.05
        disc_rate = 0.05 if qty >= 5 else 0.0

    item_disc = line_gross * disc_rate
    item_discount_total = item_discount_total + item_disc
    line_items.append([code, name, category, qty, line_gross, item_disc, tax_rate])

after_item_disc = gross_subtotal - item_discount_total

# Bill threshold discount rule (> Rs. 10,000 gets 7% extra)
if after_item_disc > 10000.0:
    threshold_disc = after_item_disc * 0.07
else:
    threshold_disc = 0.0

# Enforce maximum total concession limit of Rs. 3,000.00
max_concession = 3000.0
raw_total_disc = item_discount_total + threshold_disc
cap_applied = False
```

case_study_07.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
55
56  if raw_total_disc > max_concession:
57      cap_applied = True
58      scale_factor = max_concession / raw_total_disc
59      total_concession = max_concession
60  else:
61      scale_factor = 1.0
62      total_concession = raw_total_disc
63
64  print("\n" + "=" * 76)
65  print("                  ITEMIZED RETAIL TAX INVOICE")
66  print("=" * 76)
67  print(f"{'Code':<7}{'Item':<13}{'Cat':<13}{'Qty':<5}{'Gross':<10}{'Conc':<9}{'GST':<9}
        {'Net(Rs)'}")
68  print("-" * 76)
69
70  total_gst = 0.0
71  grand_total = 0.0
72
73  for row in line_items:
74      c_code, c_name, c_cat, c_qty, l_gross, l_disc, t_rate = row
75      share_thresh = (l_gross - l_disc) / after_item_disc * threshold_disc if after_item
        _disc > 0 else 0.0
76      effective_disc = (l_disc + share_thresh) * scale_factor
77      taxable_val = l_gross - effective_disc
78      gst_amt = taxable_val * t_rate
79      net_line = taxable_val + gst_amt
80
81      total_gst = total_gst + gst_amt
82      grand_total = grand_total + net_line
83      print(f"{c_code:<7}{c_name:<13}{c_cat:<13}{c_qty:<5}{l_gross:<10.2f}{effective_dis
        c:<9.2f}{gst_amt:<9.2f}{net_line:.2f}")
84
85  print("-" * 76)
86  cap_note = " (CAPPED AT Rs. 3000.00)" if cap_applied else ""
87  print(f"Gross Subtotal: Rs. {gross_subtotal:.2f} | Total Concession: Rs. {total_conces
        sion:.2f}{cap_note}")
88  print(f"Total GST:      Rs. {total_gst:.2f} | GRAND TOTAL PAYABLE: Rs. {grand_total:.2
        f}")
89  print("=" * 76)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-07A	Cap Triggered (>Rs.3000)	2x Monitor(14k), 3x Jacket(2.5k), 6x Rice(800) -> Gross Rs.40,300	Raw disc Rs.5844.90 capped at Rs.3000.00; GST Rs.5715.22; Net Rs.43,015.22	**[PASS] Total Concession: Rs.3000.00 (CAPPED) | Net: Rs.43015.22**
TC-07B	No Discount Small Basket	4x MilkPack(60), 2x Shirt(1200) -> Gross Rs.2,640	Concession Rs.0.00; GST Rs.300.00; Grand Total Rs.2,940.00	**[PASS] Total Concession: Rs.0.00 | Grand Total: Rs.2940.00**

Test Case 1: Multi-Category Bulk Discounts, >10k Threshold & Rs. 3000 Cap — Input Session Screenshot

Electronics (2x14000), Apparel (3x2500), Essential (6x800). Raw discounts exceed Rs. 3,000, triggering proportional Rs. 3,000 concession cap.

Terminal — Test Case 1 Input Screenshot (python3 case_study_07.py)

```
$ python3 case_study_07.py
Enter number of distinct products: 3
Product Code: EL01
Product Name: Monitor
Category (ESSENTIAL/APPAREL/ELECTRONICS): ELECTRONICS
Unit Price (INR): 14000
Quantity: 2
Product Code: AP02
Product Name: Jackets
```

Terminal — Test Case 1 Input Screenshot (python3 case_study_07.py) (continued — part 2)

```
Category (ESSENTIAL/APPAREL/ELECTRONICS): APPAREL
Unit Price (INR): 2500
Quantity: 3
Product Code: ES03
Product Name: RiceBag
Category (ESSENTIAL/APPAREL/ELECTRONICS): ESSENTIAL
Unit Price (INR): 800
Quantity: 6
```

Test Case 2: Sub-Threshold Basket Without Bulk Quantities — Input Session Screenshot

Small basket below Rs. 10,000 and below bulk quantity minimums; verifies 0% concession and accurate 5% & 12% GST computation.

Terminal — Test Case 2 Input Screenshot (python3 case_study_07.py)

```
$ python3 case_study_07.py
Enter number of distinct products: 2
Product Code: ES10
Product Name: MilkPack
Category (ESSENTIAL/APPAREL/ELECTRONICS): ESSENTIAL
Unit Price (INR): 60
Quantity: 4
Product Code: AP11
Product Name: Shirt
Category (ESSENTIAL/APPAREL/ELECTRONICS): APPAREL
Unit Price (INR): 1200
Quantity: 2
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_07.py)

```
================================================================================
                ITEMIZED RETAIL TAX INVOICE
================================================================================
Code   Item         Cat          Qty  Gross     Conc     GST      Net(Rs)
--------------------------------------------------------------------------------
EL01   Monitor      ELECTRONICS  2    28000.00  2082.37  4665.17  30582.80
AP02   Jackets      APPAREL      3    7500.00   629.62   824.45   7694.82
ES03   RiceBag      ESSENTIAL    6    4800.00   288.00   225.60   4737.59
--------------------------------------------------------------------------------
Gross Subtotal: Rs. 40300.00 | Total Concession: Rs. 3000.00 (CAPPED AT Rs. 3000.00)
Total GST:      Rs. 5715.22 | GRAND TOTAL PAYABLE: Rs. 43015.22
================================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_07.py)

```
================================================================================
                ITEMIZED RETAIL TAX INVOICE
================================================================================
Code   Item         Cat          Qty  Gross     Conc     GST      Net(Rs)
--------------------------------------------------------------------------------
ES10   MilkPack     ESSENTIAL    4    240.00    0.00     12.00    252.00
AP11   Shirt        APPAREL      2    2400.00   0.00     288.00   2688.00
--------------------------------------------------------------------------------
Gross Subtotal: Rs. 2640.00 | Total Concession: Rs. 0.00
Total GST:      Rs. 300.00 | GRAND TOTAL PAYABLE: Rs. 2940.00
================================================================================
```

10. Conclusion

Using a two-pass nested-list architecture, this retail billing engine accurately coordinates item-level bulk rules, invoice-wide threshold discounts, proportional concession capping, and multi-slab GST computation.

CASE STUDY 08 OF 10

8. Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2. Objectives

- Compute comprehensive batch production cost incorporating raw materials, percentage material wastage, labour hours, and machine hours.
- Evaluate batch revenue, net profit/loss, and return-on-cost profit margin percentages.
- Classify each production batch against a user-specified target profit margin into TARGET_MET, BELOW_TARGET, or LOSS_INCURRED.
- Aggregate plant-wide manufacturing expenditure, revenue, and target compliance counts.

3. Assumptions & Domain Rules

- Standard shop-floor overhead rates: Direct Labour = Rs. 250.00/hour; Machine Operation = Rs. 400.00/hour.
- Material Wastage Cost = (Units * Raw_Material_Per_Unit) * (Wastage_% / 100).
- Profit Margin % is defined on manufacturing cost: ((Revenue - Total_Cost) / Total_Cost) * 100.
- Classification: Profit < 0 -> LOSS_INCURRED; Margin % >= Target_Margin -> TARGET_MET; else BELOW_TARGET.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read target_margin (%) and num_batches; collect 8-element batch tuples into batches.

Step 3: Set labour_rate = 250.0, machine_rate = 400.0, overall_cost = 0, overall_rev = 0, met_count = 0.

Step 4: Unpack each batch tuple (b_id, prod, units, mat_unit, wastage_pct, labour_hrs, machine_hrs, sell_price).

Step 5: Compute base_mat_cost = units * mat_unit, wastage_cost = base_mat_cost * (wastage_pct / 100), labour_cost = labour_hrs * 250, machine_cost = machine_hrs * 400.

Step 6: Sum into total_cost; compute revenue = units * sell_price, profit = revenue - total_cost, and margin_pct = (profit / total_cost) * 100.

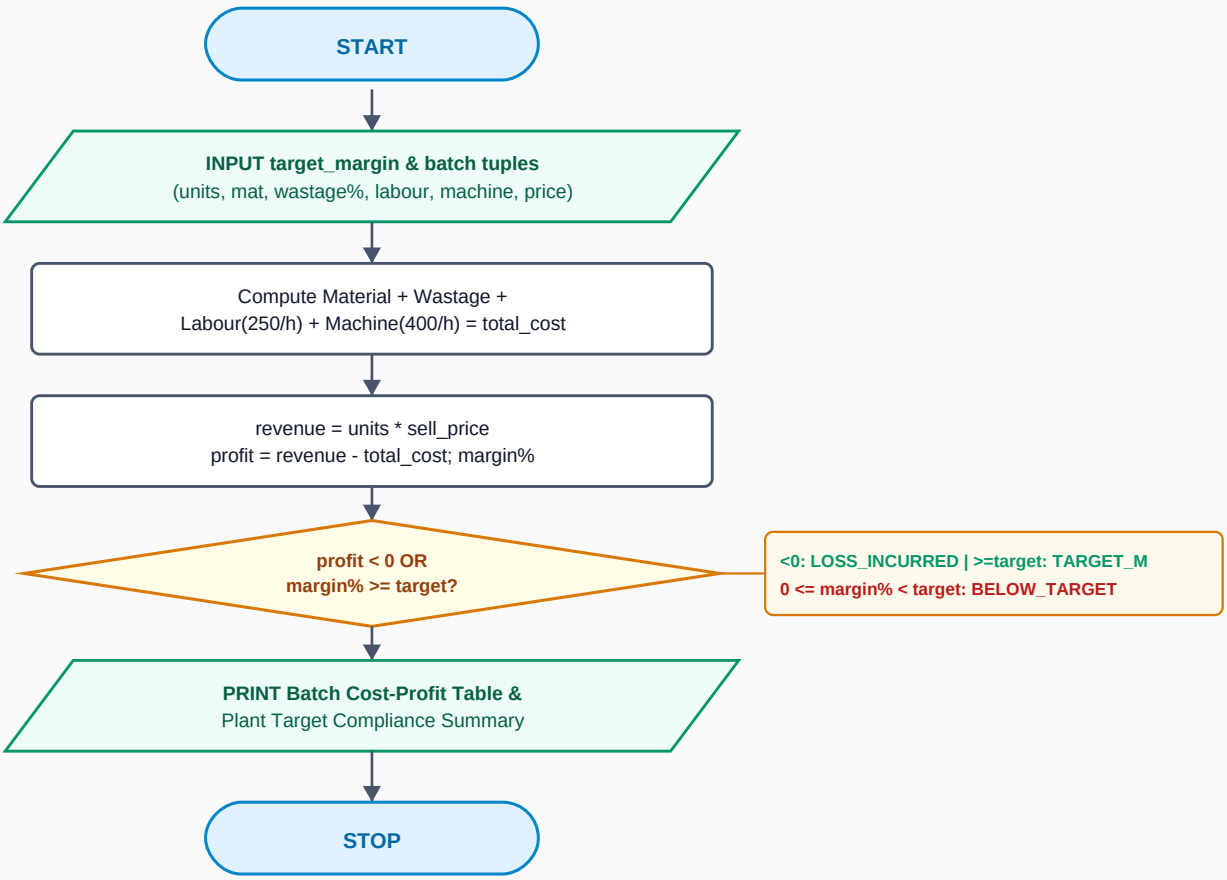
Step 7: Classify batch as LOSS_INCURRED (profit < 0), TARGET_MET (margin_pct >= target_margin, increment met_count), or BELOW_TARGET.

Step 8: Accumulate plant totals and print batch profitability table and plant summary.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 08: Manufacturing Cost and Profitability Analysis

```
BEGIN
    INPUT target_margin, num_batches
    READ batch tuples INTO batches
    SET labour_rate = 250, machine_rate = 400, met_count = 0

    FOR EACH (b_id, prod, units, mat_unit, waste_pct, l_hrs, m_hrs, price) IN batches DO
        SET base_mat = units * mat_unit
        SET waste_cost = base_mat * (waste_pct / 100.0)
        SET total_cost = base_mat + waste_cost + (l_hrs * labour_rate) + (m_hrs * machine_
    rate)
        SET revenue = units * price
        SET profit = revenue - total_cost
        SET margin_pct = (profit / total_cost) * 100.0

        IF profit < 0 THEN status = "LOSS_INCURRED"
        ELSE IF margin_pct >= target_margin THEN status = "TARGET_MET"; INCREMENT met_coun
    t
        ELSE status = "BELOW_TARGET"
        END IF
        PRINT batch analysis row
    END FOR
    PRINT overall cost, revenue, net profit, met_count
END
```

7. Python Source Code (case_study_08.py)

case_study_08.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 8: Manufacturing Cost and Profitability Analysis
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Lists, Tuples, Unpacking

target_margin = float(input("Enter Target Profit Margin (%): "))
num_batches = int(input("Enter number of production batches: "))

batches = []
for i in range(num_batches):
    b_id = input("Batch ID: ").strip()
    prod = input("Product Name: ").strip()
    units = int(input("Units Produced: "))
    mat_unit = float(input("Raw Material Cost per Unit (INR): "))
    wastage_pct = float(input("Material Wastage (%): "))
    labour_hrs = float(input("Labour Hours: "))
    machine_hrs = float(input("Machine Hours: "))
    sell_price = float(input("Selling Price per Unit (INR): "))
    batches.append((b_id, prod, units, mat_unit, wastage_pct, labour_hrs, machine_hrs, sell_price))

labour_rate = 250.0
machine_rate = 400.0

print("\n" + "=" * 78)
print(f"MANUFACTURING COST & PROFITABILITY REPORT (Target Margin: {target_margin:.1f}%)")
print("=" * 78)
print(f"{'Batch':<7}{'Product':<12}{'TotalCost':<12}{'Revenue':<12}{'Profit':<11}{'Margin%':<9}{'Status'}")
print("-" * 78)

overall_cost = 0.0
overall_rev = 0.0
met_count = 0

for batch in batches:
    b_id, prod, units, mat_unit, wastage_pct, labour_hrs, machine_hrs, sell_price = batch

    base_mat_cost = units * mat_unit
    wastage_cost = base_mat_cost * (wastage_pct / 100.0)
    labour_cost = labour_hrs * labour_rate
    machine_cost = machine_hrs * machine_rate

    total_cost = base_mat_cost + wastage_cost + labour_cost + machine_cost
    revenue = units * sell_price
    profit = revenue - total_cost

    if total_cost > 0:
        margin_pct = (profit / total_cost) * 100.0
    else:
        margin_pct = 0.0

    if profit < 0:
        status = "LOSS_INCURRED"
    elif margin_pct >= target_margin:
        status = "TARGET_MET"
        met_count = met_count + 1
    else:
        status = "BELOW_TARGET"

    overall_cost = overall_cost + total_cost
    overall_rev = overall_rev + revenue

```

case_study_08.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
60      print(f"{b_id:<7}{prod:<12}{total_cost:<12.2f}{revenue:<12.2f}{profit:<11.2f}{marg
        in_pct:<9.2f}{status}")
61
62  print("-" * 78)
63  net_overall = overall_rev - overall_cost
64  print(f"Total Cost: Rs. {overall_cost:.2f} | Total Revenue: Rs. {overall_rev:.2f} | Ne
        t Profit: Rs. {net_overall:.2f}")
65  print(f"Batches Meeting Profit Target: {met_count} of {len(batches)}")
66  print("=" * 78)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-08A	3-State Batch Classification	Target 20%: B01 GearBox, B02 ValveSet, B03 ShaftRod	B01 TARGET_MET (20.13%), B02 LOSS_INCURRED (-2160), B03 BELOW_TARGET (12.23%)	[PASS] B01 TARGET_MET | B02 LOSS_INCURRED | B03 BELOW_TARGET (1 of 3 met)
TC-08B	15% Target Precision Check	Target 15%: B04 PumpCore (4% waste), B05 SealKit (15% waste)	B04 TARGET_MET (22.09%); B05 LOSS_INCURRED due to labour/machine overhead	[PASS] B04 22.09% TARGET_MET | B05 -13.15% LOSS_INCURRED

Test Case 1: Target Met, Loss Incurred & Below-Target Batches — Input Session Screenshot

Target 20% margin across 3 batches: GearBox meets target, ValveSet incurs loss due to wastage/overhead, ShaftRod profitable but below 20%.

Terminal — Test Case 1 Input Screenshot (python3 case_study_08.py)

```
$ python3 case_study_08.py
Enter Target Profit Margin (%): 20
Enter number of production batches: 3
Batch ID: B01
Product Name: GearBox
Units Produced: 200
Raw Material Cost per Unit (INR): 320
Material Wastage (%): 5
Labour Hours: 40
Machine Hours: 25
Selling Price per Unit (INR): 520
Batch ID: B02
Product Name: ValveSet
Units Produced: 150
Raw Material Cost per Unit (INR): 180
Material Wastage (%): 8
Labour Hours: 30
Machine Hours: 15
Selling Price per Unit (INR): 270
Batch ID: B03
Product Name: ShaftRod
Units Produced: 100
Raw Material Cost per Unit (INR): 250
Material Wastage (%): 12
Labour Hours: 35
Machine Hours: 20
Selling Price per Unit (INR): 390
```

Test Case 2: High-Margin Precision Batch vs High-Wastage Batch — Input Session Screenshot

Target 15% margin across 2 batches testing wastage impact on unit profitability.

Terminal — Test Case 2 Input Screenshot (python3 case_study_08.py)

```
$ python3 case_study_08.py
Enter Target Profit Margin (%): 15
Enter number of production batches: 2
Batch ID: B04
Product Name: PumpCore
Units Produced: 120
Raw Material Cost per Unit (INR): 400
```

Terminal — Test Case 2 Input Screenshot (python3 case_study_08.py) (continued — part 2)

```
Material Wastage (%): 4
Labour Hours: 20
Machine Hours: 15
Selling Price per Unit (INR): 620
Batch ID: B05
Product Name: SealKit
Units Produced: 80
Raw Material Cost per Unit (INR): 90
Material Wastage (%): 15
Labour Hours: 18
Machine Hours: 10
Selling Price per Unit (INR): 180
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_08.py)

```
================================================================================
MANUFACTURING COST & PROFITABILITY REPORT (Target Margin: 20.0%)
================================================================================
Batch  Product     TotalCost   Revenue     Profit     Margin%  Status
--------------------------------------------------------------------------------
B01    GearBox     87200.00    104000.00   16800.00   19.27    BELOW_TARGET
B02    ValveSet    42660.00    40500.00    -2160.00   -5.06    LOSS_INCURRED
B03    ShaftRod    44750.00    39000.00    -5750.00   -12.85   LOSS_INCURRED
--------------------------------------------------------------------------------
Total Cost: Rs. 174610.00 | Total Revenue: Rs. 183500.00 | Net Profit: Rs. 8890.00
Batches Meeting Profit Target: 0 of 3
================================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_08.py)

```
================================================================================
MANUFACTURING COST & PROFITABILITY REPORT (Target Margin: 15.0%)
================================================================================
Batch  Product     TotalCost   Revenue     Profit     Margin%  Status
--------------------------------------------------------------------------------
B04    PumpCore    60920.00    74400.00    13480.00   22.13    TARGET_MET
B05    SealKit     16780.00    14400.00    -2380.00   -14.18   LOSS_INCURRED
--------------------------------------------------------------------------------
Total Cost: Rs. 77700.00 | Total Revenue: Rs. 88800.00 | Net Profit: Rs. 11100.00
Batches Meeting Profit Target: 1 of 2
================================================================================
```

10. Conclusion

This program models industrial batch cost accounting—isolating raw material wastage and shop-floor overheads—to evaluate unit economics and profit target attainment using tuple unpacking.

CASE STUDY 09 OF 10

9. Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert travel expenses using supplied exchange rates, incorporate conversion fees, compare planned spending against a budget, and identify the largest expense category.

2. Objectives

- Maintain foreign exchange rates and forex conversion fee percentages in an immutable list of tuples.
- Convert multi-currency planned travel expenses (USD, EUR, GBP, JPY, INR) into INR inclusive of currency-specific conversion fees.
- Aggregate spending by category using a dynamic nested list [[category, sum_inr], ...] without using Python dictionaries.
- Compare total planned INR expenditure against budget to report surplus/deficit and identify the single largest expense category.

3. Assumptions & Domain Rules

- Supplied Exchange Tuple Table: USD (84.00, 2.5% fee), EUR (91.50, 2.0% fee), GBP (108.00, 2.5% fee), JPY (0.56, 3.0% fee), INR (1.00, 0.0% fee).
- Total INR Cost per item = (Foreign_Amount * Exchange_Rate) * (1 + Fee_% / 100).
- Category totals are accumulated in a 2-element nested list [category_name, total_inr] to remain strictly within list/tuple syllabus.
- Budget status is WITHIN BUDGET when Planned_Spend <= Budget_INR, and OVER BUDGET otherwise.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read budget_inr and num_expenses; initialize exchange rate tuple list rates and empty nested list category_totals = [].

Step 3: Read num_expenses tuples (cat, desc, curr, amt) into expenses.

Step 4: For each expense tuple, search rates via loop to unpack matching (r_code, ex_rate, fee_pct).

Step 5: Compute base_inr = amt * ex_rate, conv_fee = base_inr * (fee_pct / 100), and final_inr = base_inr + conv_fee; add to grand_total_inr.

Step 6: Search category_totals for cat: if found, add final_inr to pair[1]; if not found, append [cat, final_inr].

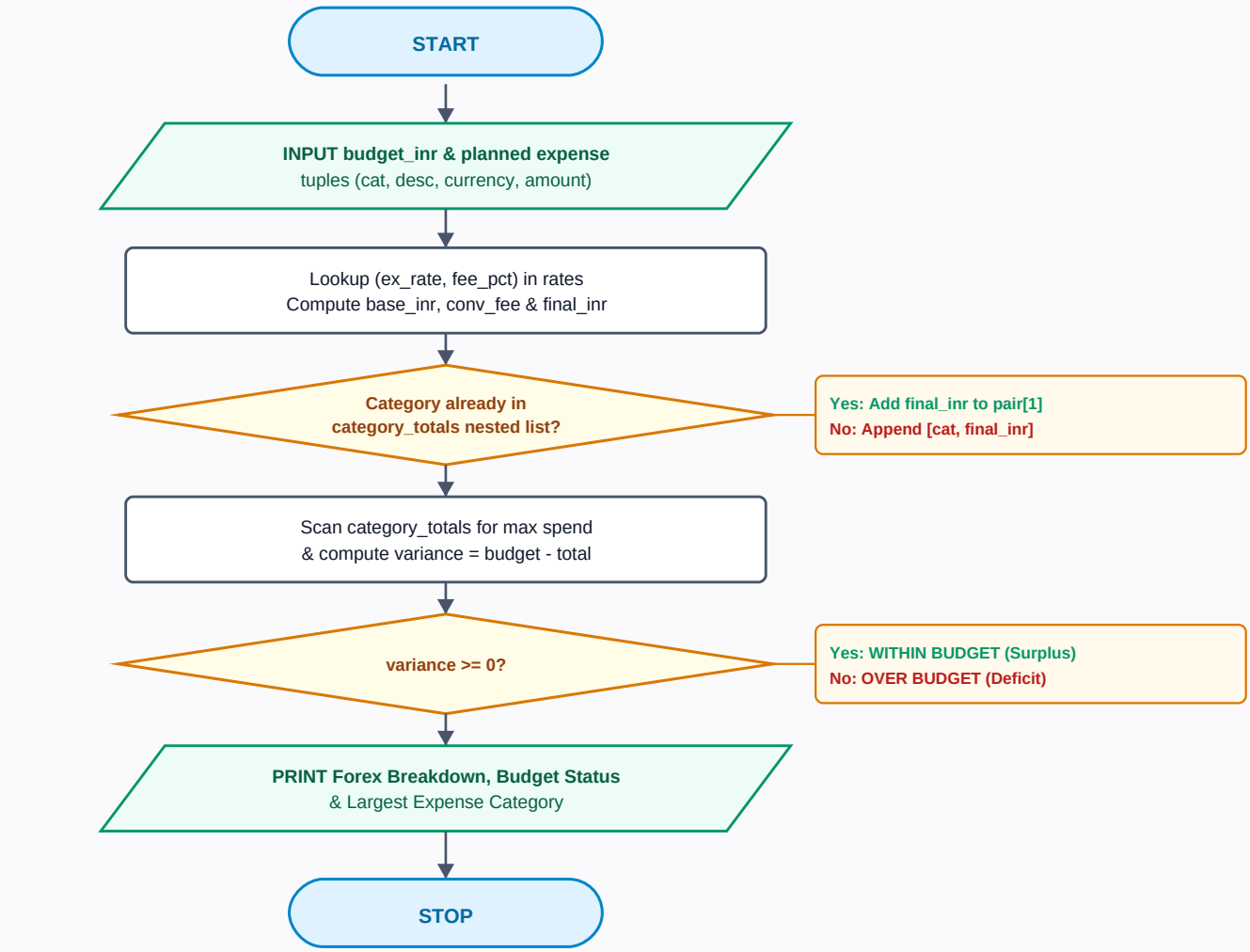
Step 7: Traverse category_totals to find top_category with maximum accumulated INR spend and compute its percentage share.

Step 8: Compute variance = budget_inr - grand_total_inr and print itemized conversion table, budget variance, and largest category.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 09: Multi-Currency Travel Budget Analysis

```
BEGIN
    INPUT budget_inr, num_expenses
    INITIALIZE rates = [("USD",84.0,2.5), ("EUR",91.5,2.0), ("GBP",108.0,2.5), ("JPY",0.56
    ,3.0), ("INR",1.0,0.0)]
    SET category_totals = [], grand_total = 0

    FOR EACH (cat, desc, curr, amt) IN expenses DO
        LOOKUP (ex_rate, fee_pct) FOR curr IN rates
        SET base_inr = amt * ex_rate
        SET conv_fee = base_inr * (fee_pct / 100.0)
        SET final_inr = base_inr + conv_fee
        SET grand_total = grand_total + final_inr

        IF cat IN category_totals THEN ADD final_inr TO matching [cat, sum]
        ELSE APPEND [cat, final_inr] TO category_totals
        END IF
        PRINT converted expense row
    END FOR

    FIND [top_category, top_amount] WITH MAXIMUM sum IN category_totals
```

PSEUDOCODE — Case Study 09: Multi-Currency Travel Budget Analysis (continued — part 2)

```
    COMPARE grand_total AGAINST budget_inr AND PRINT summary
END
```

7. Python Source Code (case_study_09.py)

case_study_09.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 9: Multi-Currency Travel Budget Analysis
# Syllabus Concepts: Variables, Operators, Conditionals, Nested Loops, Strings, Nested Lists, Tuples

budget_inr = float(input("Enter Total Travel Budget (INR): "))
num_expenses = int(input("Enter number of planned expense entries: "))

# Exchange rate table: (Currency, INR Rate, Conversion Fee %)
rates = [
    ("USD", 84.00, 2.5),
    ("EUR", 91.50, 2.0),
    ("GBP", 108.00, 2.5),
    ("JPY", 0.56, 3.0),
    ("INR", 1.00, 0.0)
]

expenses = []
for i in range(num_expenses):
    cat = input("Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): ").strip().upper()
    desc = input("Expense Description: ").strip()
    curr = input("Currency (USD/EUR/GBP/JPY/INR): ").strip().upper()
    amt = float(input("Amount in Foreign Currency: "))
    expenses.append((cat, desc, curr, amt))

category_totals = []
grand_total_inr = 0.0

print("\n" + "=" * 76)
print("             MULTI-CURRENCY TRAVEL BUDGET ANALYSIS REPORT")
print("=" * 76)
print(f"{'Category':<12}{'Description':<15}{'Foreign':<13}{'Base INR':<11}{'Fee(INR)':<10}{'Total INR'}")
print("-" * 76)

for exp in expenses:
    cat, desc, curr, amt = exp
    ex_rate = 1.0
    fee_pct = 0.0
    for r_tuple in rates:
        r_code, r_val, r_fee = r_tuple
        if r_code == curr:
            ex_rate = r_val
            fee_pct = r_fee
            break

    base_inr = amt * ex_rate
    conv_fee = base_inr * (fee_pct / 100.0)
    final_inr = base_inr + conv_fee
    grand_total_inr = grand_total_inr + final_inr

    # Update nested list [category, accumulated_inr] without dictionaries
    found_cat = False
    for pair in category_totals:
        if pair[0] == cat:
            pair[1] = pair[1] + final_inr
            found_cat = True
            break
    if not found_cat:
```

case_study_09.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
        category_totals.append([cat, final_inr])

    f_label = f"{amt:.1f} {curr}"
    print(f"{cat:<12}{desc:<15}{f_label:<13}{base_inr:<11.2f}{conv_fee:<10.2f}{final_inr:.2f}")

# Find largest expense category
top_category = "NONE"
top_amount = 0.0
for pair in category_totals:
    c_name, c_sum = pair
    if c_sum > top_amount:
        top_amount = c_sum
        top_category = c_name

print("-" * 76)
variance = budget_inr - grand_total_inr
if variance >= 0:
    b_status = f"WITHIN BUDGET (Surplus: Rs. {variance:.2f})"
else:
    b_status = f"OVER BUDGET (Exceeded by: Rs. {-variance:.2f})"

share_pct = (top_amount / grand_total_inr * 100.0) if grand_total_inr > 0 else 0.0
print(f"Planned Spend: Rs. {grand_total_inr:.2f} | Budget: Rs. {budget_inr:.2f} -> {b_status}")
print(f"Largest Expense Category: {top_category} (Rs. {top_amount:.2f}, {share_pct:.1f}% of spend)")
print("=" * 76)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-09A	4-Currency Within Budget	Budget 1.85L: 950 USD, 620 EUR, 180 GBP, 22000 JPY	Total Rs.172,499.40 -> Surplus Rs.12,500.60; Largest: FLIGHTS (47.4%)	**[PASS] Spend Rs.172499.40 | Surplus Rs.12500.60 | Top: FLIGHTS**
TC-09B	Budget Deficit Alert	Budget 90k: Flights 700 USD, Hotel 380 EUR, Activities 120 USD	Total Rs.106,053.40 -> OVER BUDGET by Rs.16,053.40	**[PASS] OVER BUDGET (Exceeded by: Rs. 16053.40)**

Test Case 1: 4 Foreign Currencies Within Budget & Dominant Category — Input Session Screenshot

USD, EUR, GBP, and JPY expenses with conversion fees against Rs. 1,85,000 budget; identifies FLIGHTS as largest category.

Terminal — Test Case 1 Input Screenshot (python3 case_study_09.py)

```
$ python3 case_study_09.py
Enter Total Travel Budget (INR): 185000
Enter number of planned expense entries: 4
Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): FLIGHTS
Expense Description: Airfare
Currency (USD/EUR/GBP/JPY/INR): USD
Amount in Foreign Currency: 950
Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): HOTEL
Expense Description: Stay4Nights
Currency (USD/EUR/GBP/JPY/INR): EUR
Amount in Foreign Currency: 620
Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): FOOD
Expense Description: Dining
Currency (USD/EUR/GBP/JPY/INR): GBP
Amount in Foreign Currency: 180
Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): TRANSIT
Expense Description: RailPass
Currency (USD/EUR/GBP/JPY/INR): JPY
Amount in Foreign Currency: 22000
```

Test Case 2: Budget Deficit Detection & Repeated Category Aggregation — Input Session Screenshot

USD appears in two different categories (FLIGHTS and ACTIVITIES) against tight Rs. 90,000 budget, triggering OVER BUDGET alert.

Terminal — Test Case 2 Input Screenshot (python3 case_study_09.py)

```
$ python3 case_study_09.py
Enter Total Travel Budget (INR): 90000
Enter number of planned expense entries: 3
Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): FLIGHTS
Expense Description: Ticket
Currency (USD/EUR/GBP/JPY/INR): USD
Amount in Foreign Currency: 700
Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): HOTEL
Expense Description: Hostel
Currency (USD/EUR/GBP/JPY/INR): EUR
Amount in Foreign Currency: 380
Category (FLIGHTS/HOTEL/FOOD/TRANSIT/ACTIVITIES): ACTIVITIES
Expense Description: MuseumTour
Currency (USD/EUR/GBP/JPY/INR): USD
Amount in Foreign Currency: 120
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_09.py)

```
================================================================================
            MULTI-CURRENCY TRAVEL BUDGET ANALYSIS REPORT
================================================================================
Category    Description    Foreign      Base INR   Fee(INR)  Total INR
--------------------------------------------------------------------------------
FLIGHTS     Airfare        950.0 USD    79800.00   1995.00   81795.00
HOTEL       Stay4Nights    620.0 EUR    56730.00   1134.60   57864.60
FOOD        Dining         180.0 GBP    19440.00   486.00    19926.00
TRANSIT     RailPass       22000.0 JPY  12320.00   369.60    12689.60
--------------------------------------------------------------------------------
Planned Spend: Rs. 172275.20 | Budget: Rs. 185000.00 -> WITHIN BUDGET (Surplus: Rs. 12724.
    80)
Largest Expense Category: FLIGHTS (Rs. 81795.00, 47.5% of spend)
================================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_09.py)

```
================================================================================
            MULTI-CURRENCY TRAVEL BUDGET ANALYSIS REPORT
================================================================================
Category    Description    Foreign      Base INR   Fee(INR)  Total INR
--------------------------------------------------------------------------------
FLIGHTS     Ticket         700.0 USD    58800.00   1470.00   60270.00
HOTEL       Hostel         380.0 EUR    34770.00   695.40    35465.40
ACTIVITIES  MuseumTour     120.0 USD    10080.00   252.00    10332.00
--------------------------------------------------------------------------------
Planned Spend: Rs. 106067.40 | Budget: Rs. 90000.00 -> OVER BUDGET (Exceeded by: Rs. 16067
    .40)
Largest Expense Category: FLIGHTS (Rs. 60270.00, 56.8% of spend)
================================================================================
```

10. Conclusion

By pairing a tuple exchange-rate lookup table with a dictionary-free nested list accumulator, this analyzer accurately tracks forex conversion fees, budget variance, and category dominance.

CASE STUDY 10 OF 10

10. Campus Event Financial Planning

1. Problem Statement

Calculate venue, equipment, catering, transport, and certificate costs; determine participant contributions, sponsorship shortfalls, and the minimum registration count needed to break even.

2. Objectives

- Separate campus event budgets into Fixed Overheads (venue, equipment, transport) and Variable Per-Participant Costs (catering, certificates).
- Aggregate confirmed sponsorship pledges from sponsor tuples alongside expected registration fee collections.
- Determine net financial surplus or exact sponsorship shortfall required to balance the event budget.
- Compute the exact integer minimum registration count required to break even using a syllabus-compliant while-loop without math.ceil.

3. Assumptions & Domain Rules

- Fixed Cost = Venue + Equipment + Transport; Variable Cost per Head = Catering_Per_Head + Certificate_Per_Head.
- Total Revenue = (Registration_Fee * Expected_Participants) + Sum(Sponsor_Pledges).
- If Total Revenue < Total Expenditure, the difference is reported as Sponsorship Shortfall.
- Break-Even Registration Count is the smallest non-negative integer N where (N * (Reg_Fee - Variable_Per_Head)) >= (Fixed_Cost - Sponsorship); infeasible if Reg_Fee <= Variable_Per_Head and unbacked fixed cost > 0.

4. Step-by-Step Algorithm

Step 1: Start the program.

Step 2: Read event_name, fixed costs (venue, equip, transport), variable costs (catering_head, cert_head), reg_fee, expected_reg, and sponsor tuples.

Step 3: Compute fixed_cost = venue + equip + transport and variable_per_head = catering_head + cert_head.

Step 4: Compute total_expense = fixed_cost + (variable_per_head * expected_reg).

Step 5: Unpack sponsor tuples to sum sponsorship_total; compute total_income = (reg_fee * expected_reg) + sponsorship_total and net_balance = total_income - total_expense.

Step 6: Compute margin_per_head = reg_fee - variable_per_head and unbacked_fixed = fixed_cost - sponsorship_total.

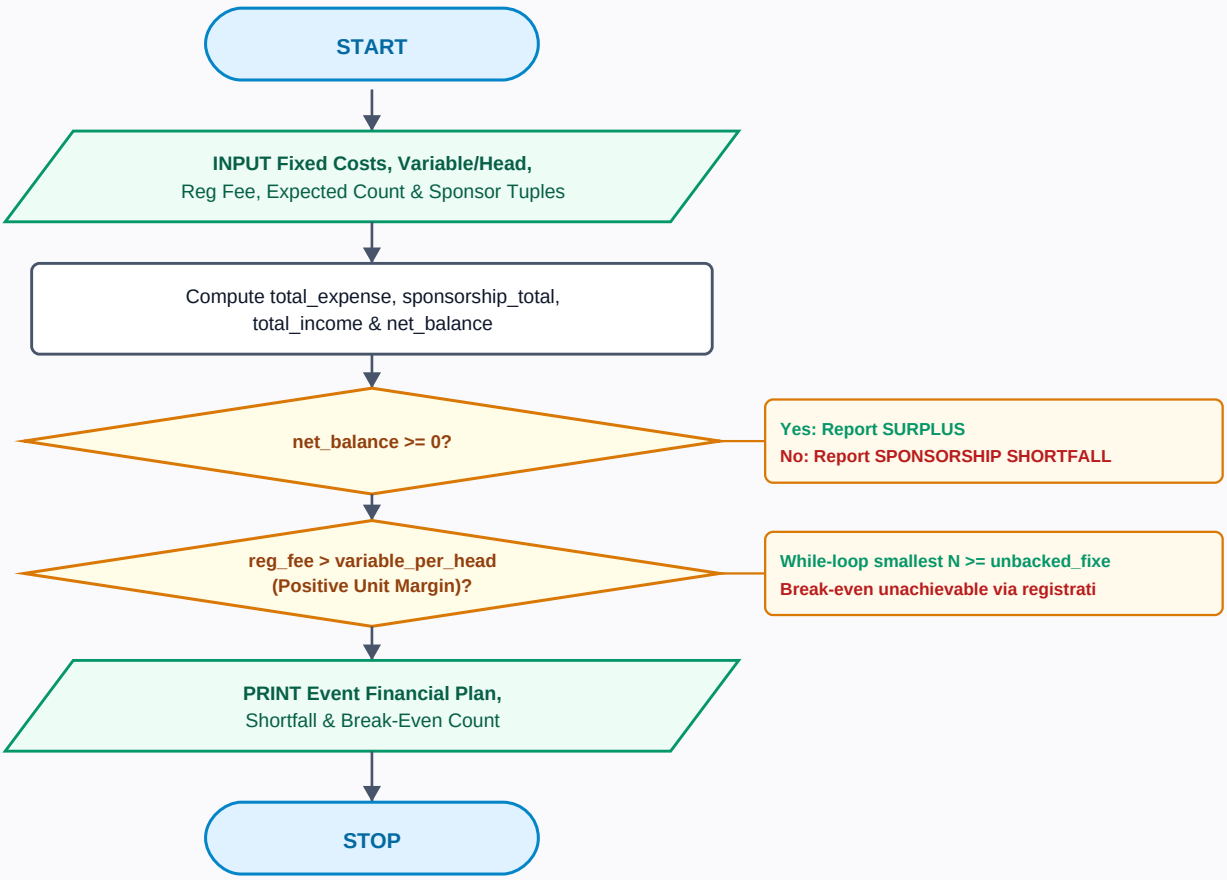
Step 7: If unbacked_fixed <= 0, breakeven_count = 0. Else if margin_per_head <= 0, mark break-even unachievable. Else increment breakeven_count from 0 in a while loop until (breakeven_count * margin_per_head) >= unbacked_fixed.

Step 8: Print itemized financial summary, surplus/shortfall status, and minimum break-even registrations.

Step 9: Stop.

5. System Flowchart (ANSI Standard)

ANSI Symbols: Oval = Start/Stop | Parallelogram = Input/Output | Rectangle = Process | Diamond = Decision



6. Structured Pseudocode

PSEUDOCODE — Case Study 10: Campus Event Financial Planning

```
BEGIN
    INPUT event_name, venue, equip, transport, catering, cert, reg_fee, expected_reg
    READ sponsor tuples INTO sponsors

    SET fixed_cost = venue + equip + transport
    SET var_head = catering + cert
    SET total_expense = fixed_cost + (var_head * expected_reg)
    SET sponsorship = SUM OF sp_amt IN sponsors
    SET total_income = (reg_fee * expected_reg) + sponsorship
    SET net_balance = total_income - total_expense

    SET margin = reg_fee - var_head
    SET unbacked = fixed_cost - sponsorship
    IF unbacked <= 0 THEN breakeven = 0
    ELSE IF margin <= 0 THEN breakeven = INFEASIBLE
    ELSE
        SET breakeven = 0
        WHILE (breakeven * margin) < unbacked DO
            SET breakeven = breakeven + 1
        END WHILE
    END IF

    PRINT Financial Statement, Surplus/Shortfall, AND breakeven
END
```

7. Python Source Code (case_study_10.py)

case_study_10.py — Strictly Syllabus-Compliant Python 3

```
# Case Study 10: Campus Event Financial Planning
# Syllabus Concepts: Variables, Operators, Conditionals, Loops, Strings, Lists, Tuples, Unpacking

event_name = input("Enter Campus Event Name: ").strip()
venue_cost = float(input("Venue Cost (INR): "))
equip_cost = float(input("Equipment & Audio-Visual Cost (INR): "))
transport_cost = float(input("Transport & Logistics Cost (INR): "))
catering_head = float(input("Catering Cost per Participant (INR): "))
cert_head = float(input("Certificate & Kit Cost per Participant (INR): "))
reg_fee = float(input("Registration Fee per Participant (INR): "))
expected_reg = int(input("Expected Participant Count: "))
num_sponsors = int(input("Number of Sponsors: "))

sponsors = []
for i in range(num_sponsors):
    s_name = input("Sponsor Name: ").strip()
    s_amt = float(input("Sponsorship Amount (INR): "))
    sponsors.append((s_name, s_amt))

fixed_cost = venue_cost + equip_cost + transport_cost
variable_per_head = catering_head + cert_head
total_variable_cost = variable_per_head * expected_reg
total_expense = fixed_cost + total_variable_cost

participant_revenue = reg_fee * expected_reg
sponsorship_total = 0.0
for sp in sponsors:
    sp_name, sp_amt = sp
    sponsorship_total = sponsorship_total + sp_amt

total_income = participant_revenue + sponsorship_total
net_balance = total_income - total_expense

# Determine minimum registration count needed to break even
margin_per_head = reg_fee - variable_per_head
unbacked_fixed = fixed_cost - sponsorship_total

if unbacked_fixed <= 0:
    breakeven_count = 0
    be_possible = True
elif margin_per_head <= 0:
    breakeven_count = -1
    be_possible = False
else:
    be_possible = True
    breakeven_count = 0
    while (breakeven_count * margin_per_head) < unbacked_fixed:
        breakeven_count = breakeven_count + 1

print("\n" + "=" * 72)
print(f"CAMPUS EVENT FINANCIAL PLAN: {event_name.upper()}")
print("=" * 72)
print(f"Fixed Costs (Venue + Equip + Transport) : Rs. {fixed_cost:10.2f}")
print(f"Variable Cost ({expected_reg} x Rs. {variable_per_head:.2f}/head)   : Rs. {total_variable_cost:10.2f}")
print(f"Total Planned Expenditure              : Rs. {total_expense:10.2f}")
print("-" * 72)
print(f"Participant Contributions ({expected_reg} regs)   : Rs. {participant_revenue:10.2f}")
print(f"Confirmed Sponsorship ({len(sponsors)} sponsors)      : Rs. {sponsorship_total:10.2f}")
print(f"Total Projected Revenue                : Rs. {total_income:10.2f}")
print("-" * 72)
```

case_study_10.py — Strictly Syllabus-Compliant Python 3 (continued — part 2)

```
61
62  if net_balance >= 0:
63      print(f"Financial Status    : SURPLUS of Rs. {net_balance:.2f} (No Sponsorship Sho
        rtfall)")
64  else:
65      print(f"Financial Status    : SPONSORSHIP SHORTFALL of Rs. {-net_balance:.2f}")
66
67  if be_possible:
68      print(f"Break-Even Count    : Minimum {breakeven_count} registrations required")
69  else:
70      print("Break-Even Count    : Not achievable (Registration fee <= variable cost/hea
        d)")
71  print("=" * 72)
```

8. Test Cases Matrix & Test Case Input Screenshots

TC ID	Scenario	Test Input Summary	Expected Output	Verified Result
TC-10A	Surplus & Exact Break-Even	Fixed 60k, Var 180/hd, Fee 350/hd, 160 regs, Sponsors 40k	Expense 88.8k, Revenue 96k -> Surplus Rs.7,200; Break-Even = 118 regs	[PASS] SURPLUS Rs.7200.00 | Minimum 118 registrations required
TC-10B	Negative Margin Shortfall	Fixed 25k, Var 230/hd, Fee 200/hd, 80 regs, Sponsor 5k	Shortfall Rs.22,400; Break-even unachievable (Fee <= Var/head)	[PASS] SHORTFALL Rs.22400.00 | Break-Even: Not achievable

Test Case 1: Profitable TechFest with Multi-Sponsor Backing — Input Session Screenshot

Fixed Rs. 60,000, Variable Rs. 180/head, Fee Rs. 350/head, 160 participants, Rs. 40,000 sponsorship -> Surplus & exact break-even count.

Terminal — Test Case 1 Input Screenshot (python3 case_study_10.py)

```
$ python3 case_study_10.py
Enter Campus Event Name: TechFest 2026
Venue Cost (INR): 35000
Equipment & Audio-Visual Cost (INR): 18000
Transport & Logistics Cost (INR): 7000
Catering Cost per Participant (INR): 140
Certificate & Kit Cost per Participant (INR): 40
Registration Fee per Participant (INR): 350
Expected Participant Count: 160
Number of Sponsors: 2
Sponsor Name: CodeCorp
Sponsorship Amount (INR): 25000
Sponsor Name: CloudLabs
Sponsorship Amount (INR): 15000
```

Test Case 2: Subsidized Workshop with Unviable Per-Head Fee — Input Session Screenshot

Registration fee (Rs. 200) is lower than variable cost per head (Rs. 230), demonstrating sponsorship shortfall and unachievable break-even.

Terminal — Test Case 2 Input Screenshot (python3 case_study_10.py)

```
$ python3 case_study_10.py
Enter Campus Event Name: AI Workshop
Venue Cost (INR): 15000
Equipment & Audio-Visual Cost (INR): 8000
Transport & Logistics Cost (INR): 2000
Catering Cost per Participant (INR): 180
Certificate & Kit Cost per Participant (INR): 50
Registration Fee per Participant (INR): 200
Expected Participant Count: 80
Number of Sponsors: 1
Sponsor Name: LocalBookStore
Sponsorship Amount (INR): 5000
```

9. Execution Output Screenshots

Terminal — Test Case 1 Verified Output Screenshot (case_study_10.py)

```
======================================================================
CAMPUS EVENT FINANCIAL PLAN: TECHFEST 2026
======================================================================
Fixed Costs (Venue + Equip + Transport) : Rs.   60000.00
Variable Cost (160 x Rs. 180.00/head)   : Rs.   28800.00
Total Planned Expenditure               : Rs.   88800.00
----------------------------------------------------------------------
Participant Contributions (160 regs)   : Rs.   56000.00
Confirmed Sponsorship (2 sponsors)      : Rs.   40000.00
Total Projected Revenue                 : Rs.   96000.00
----------------------------------------------------------------------
Financial Status    : SURPLUS of Rs. 7200.00 (No Sponsorship Shortfall)
Break-Even Count    : Minimum 118 registrations required
======================================================================
```

Terminal — Test Case 2 Verified Output Screenshot (case_study_10.py)

```
======================================================================
CAMPUS EVENT FINANCIAL PLAN: AI WORKSHOP
======================================================================
Fixed Costs (Venue + Equip + Transport) : Rs.   25000.00
Variable Cost (80 x Rs. 230.00/head)   : Rs.   18400.00
Total Planned Expenditure               : Rs.   43400.00
----------------------------------------------------------------------
Participant Contributions (80 regs)   : Rs.   16000.00
Confirmed Sponsorship (1 sponsors)      : Rs.    5000.00
Total Projected Revenue                 : Rs.   21000.00
----------------------------------------------------------------------
Financial Status    : SPONSORSHIP SHORTFALL of Rs. 22400.00
Break-Even Count    : Not achievable (Registration fee <= variable cost/head)
======================================================================
```

10. Conclusion

The Campus Event Financial Planner integrates fixed/variable cost decomposition, sponsor tuple aggregation, and while-loop integer break-even analysis with negative-margin infeasibility guards.